GAPMAP

Development Report

Tayef Wasee – 33849950 & Ismaeel Khan - 33819922

IS2018H: Computing Project 2 (2025-26)

CONTENTS

DETAILS 4
1 INTRODUCTION 5
1.1 Project Overview 5
1.2 Problem Statement 6
1.3 Aims and Objectives 7
1.4 Project Scope 8
1.5 Report Overview 10
2 PLANNING, RESEARCH AND REQUIREMENTS 11
2.1 Initial Research 11
2.2 User and Stakeholder Considerations 13
2.3 Functional Requirements 14
2.4 Non-Functional Requirements 16
2.4.1 Usability requirements 18
2.4.2 Reliability requirements 18
2.4.3 Maintainability requirements 18
2.4.4 Performance requirements 18
2.4.5 Responsiveness requirements 18
2.5 Project Planning 19
2.5.1 Stage 1: define problem, narrow scope and describe MVP 19
2.5.2 Stage 2: design the database schema and backend skeleton 19
2.5.3 Stage 3: implement timetable and task CRUD foundations 20
2.5.4 Stage 4: build the recommendation engine 20
2.5.5 Stage 5: connect frontend pages, forms and API calls 20
2.5.6 Stage 6: improve validation, recommendation realism and layout polish 20
2.6 Development Methodology 22
3 PROTOTYPING AND ITERATION 25
3.1 Early Concepts 25
3.2 Low-Fidelity Prototype 26
3.3 Medium-Fidelity Prototype 27
3.4 High-Fidelity / Final Prototype 29

3.5 Iterative Changes 31
4 DESIGN SPECIFICATION 33
4.1 Final System Overview 33
4.2 System Architecture 35
4.3 Interface design 38
4.4 Data Design 39
4.5 Design Rules and Constraints 40
5 SYSTEM DEVELOPMENT 43
5.1 Front-end Development 43
5.2 Back-end Development 44
5.3 Validation and Error Handling 47
5.4 Key Features Implemented 48
5.4.1 Task creation and management 48
5.4.2 Timetable management 49
5.4.3 Recommendation engine 50
5.4.4 Dashboard summary 51
5.5 Version Control and Workflow 53
6 TESTING 55
6.1 Testing Strategy 55
6.2 Functional Testing 57
6.3 Validation Testing 59
6.4 UI and Usability Testing 61
6.5 Test Results and Fixes 63
6.6 Summary of Testing 64
7 ANALYSIS 66
7.1 How well GapMap solves the problem 66
7.2 Analysis of design decisions 67
7.3 Technical challenges and difficulties 68
8 EVALUATION 71
8.1 Evaluation of the final product 71
8.2 Evaluation of the development process 73
8.3 Limitations 75
8.4 Future Improvements 76

9 CONCLUSION 79
10 USER GUIDE 80
10.1 Accessing the system 80
10.1.1 Running the back end of the system 80
10.1.2 Running the front end of the system 81
10.2 Creating a Task 82
10.3 Editing and deleting tasks 83
10.4 Creating a timetable entry 84
10.5 Adjusting the active day window 85
10.6 Dashboard 86
10.7 Recommendations 87
10.8 Common User Notes 88
11 REFERENCES 89

DETAILS

Student(s)	Tayef Wasee – 33849950 – twase001@gold.ac.uk Ismaeel Khan – 33819922 – ikhan004@gold.ac.uk
Module	Computing Project 2
Tutor	Sean McGrath
GitHub Repository	github.com/Tayef006/GapMap
Technology Stack	React, Vite, Node.js, Express and SQLite

1 INTRODUCTION

1.1 Project Overview

GapMap is a web-based productivity tool that combines task management, deadline tracking, and schedule planning into one system to help users better arrange their professional or academic obligations. Its main goal is to help users make better use of the few spare moments that frequently arise in between planned activities. GapMap integrates these components in a more useful and clever manner than traditional productivity applications, which only list activities or show events separately. By doing this, it enables users to document their tasks whilst they determine when they can really finish them given their daily schedule.

The system was primarily created for university students who frequently have erratic weekly schedules, concurrent coursework deadlines and fragmented time between lectures, seminars, lab sessions, part-time jobs and other personal obligations. Because traditional task managers do not consider timetable structure and calendar applications do not actively recommend appropriate work to accomplish during those periods, students find it difficult to effective use of these brief gaps. In order to solve this problem GapMap develops a more integrated environment that allows for the simultaneous management of activities and planned sessions. Users can now more easily arrange their workload in a way that better reflects the realities of student life.

The application is primarily intended for students, although the basic idea is equally applicable to a broader audience. A system that assists in identifying valuable work that can be finished in constrained windows of availability may also be beneficial to early-career professionals and other users with meeting-based schedules. However, the current version of GapMap was created with university students' typical workflow, stresses and time management issues in mind. This emphasis affected the application's feature set as well as its overall design.

Furthermore, GapMap's recommendation-based methodology is a crucial aspect that sets it apart from a typical to-do list. In addition to storing tasks and sessions, the program evaluates the user's schedule to identify any open gaps and then suggests tasks that are appropriate for the duration and timing of those gaps. For instance, a brief administrative chore or revising exercise might be recommended for a 20-minute break, whereas longer free periods could be paired with more difficult assignments or coursework. Because the system assists the user in making decisions about how to spend their time rather than relying on them to manually interpret their schedule, it is more useful and supportive than typical productivity programs.

With all aspects considered, GapMap may be characterised as a targeted productivity tool that attempts to enhance time management, cut down on

unnecessary downtime, and facilitate more practical planning. The application provides students with a more organised and intentional method of managing their academic workload by fusing the features of a task organiser, schedule planner, and lightweight recommendation engine. Its design embodies the notion that being productive involves not just understanding what needs to be done but also figuring out when it makes the most sense to accomplish it.

1.2 Problem Statement

Although students are typically aware of their upcoming assignments, revision tasks and coursework deadlines, the time available to complete this work is frequently divided into short and irregular periods. This common and practical issue with student productivity served as the driving force behind the creation of GapMap. These gaps can arise between lectures, seminars, lab sessions, study blocks, or personal obligations, but they are often underutilised since students find it difficult to determine what can actually be accomplished in that short amount of time. As a result, even when there is crucial work that may be advanced, vital times of availability are frequently squandered.

Students' tendency to handle their assignments using a variety of disparate techniques and resources is a significant contributing factor to this issue. Important deadlines can be recorded into distinct digital programs, written down in notebooks, stored in phone reminders, addressed in texts, or consciously recalled. This disorganised approach makes it more difficult to keep a clear picture of what needs to be done and impairs deadline awareness. Students are more inclined to neglect work, misjudge urgency, or put off deciding how to start when assignments are not arranged into a single system.

Another problem is that the majority of current productivity solutions only offer a restricted range of support. While generic to-do list programs are useful for organising and documenting tasks, they seldom assist the user in deciding if a given job is appropriate for a given period of time. Calendar and timetable systems, on the other hand, are helpful for indicating when a user is busy, but they usually don't suggest what should be done during the downtime between those obligations. As a result, students frequently have two incomplete perspectives of their workload: one indicates what needs to be done, while the other indicates when they are busy. However, neither of these perspectives enables them to make a meaningful connection between the two.

Practical decision-making is hampered by this lack of integration. In addition to knowing what assignments are available and when they have free time, students also require assistance in determining how best to manage a limited amount of time. While a lengthier 60-minute gap might be better suitable for serious work like writing, problem-solving, or revising, a student with a 20-minute gap might benefit from

checking notes, responding to emails, or making small progress on an assignment. Making these choices on a regular basis can become exhausting and ineffective without a system that actively takes into account both work needs and timetable structure, especially when the user is already under pressure from several deadlines.

Also, the fact that setting priorities becomes more difficult when mental energy is low exacerbates the issue. Not because they don't want to work, but rather because they are overburdened by conflicting deadlines, inadequate information, and the work involved in determining what is feasible at any given time, students frequently choose poorly. In these circumstances, uncertainty, procrastination, or selecting a work that is too big for the available time can waste even brief periods of spare time.

current methods are consequently not optimal, because they keep scheduling and action selection apart. The practical dilemma of how to spend the next twenty, thirty, or sixty minutes is not immediately addressed by either a schedule or a to-do list, although the former may help identify unfinished tasks and the latter may show when free time is available. In order to close this gap, GapMap was created. It does this by connecting jobs and timetable data, then utilising that data to suggest reasonable work for open time periods.

The productivity and well-being of students can be significantly impacted by making better use of fragmented time, which makes this issue worthwhile. Procrastination may be decreased, deadlines can be met more quickly, and students will feel more in control of their task if even small changes are made to how short free periods are used. These minor improvements can add up to a substantial increase in confidence, efficiency, and organization over the course of a week or term. Thus, GapMap was developed to offer a more sensible and useful solution to an issue that many students deal with on a regular basis.

1.3 Aims and Objectives

The project’s main goal was to create a workable full-stack online application that enhances time management by integrating intelligent suggestion features, task storage and timetable administration into a single system. The goal was to develop an application that would not only assist users in keeping track of their obligations but also assist them in making sensible choices regarding how to use the brief windows of spare time that arise during the day. By developing a more integrated and context-aware productivity tool, the project sought to overcome the shortcomings of conventional task managers.

During the system's development, several particular goals were set in order to accomplish this primary goal. In order to guarantee that the finished product would be both technically sound and useful for the intended user, these goals concentrated on both functional and usability requirements.

The objectives were to:

- design and build a usable task management and timetable system
- allow users to add, view and remove tasks
- allow users to add, view and remove timetable sessions
- support deadline-based organisation of work
- prevent invalid task input, including deadlines earlier than the current date
- detect meaningful free gaps between timetable sessions
- recommend tasks that fit those gaps realistically
- create a clean and intuitive interface that is easy to learn
- integrate the frontend, backend and persistent storage into one working system
- test the system thoroughly and refine it in response to faults and usability issues

These goals gave the development process a defined framework and ensured that the application met the technical requirements for a full-stack web project as well as the practical issue stated in the problem statement. They worked together to direct the development of a system that was not only practical but also pertinent, dependable, and user centred.

1.4 Project Scope

In order to guarantee that the project remained feasible within the allocated time while still exhibiting a significant degree of technological complexity, the scope of GapMap was purposefully specified in a targeted and controlled manner. Scope control is crucial to software development because it keeps a project from being overly ambitious, challenging to finish, or inconsistent in quality. Because of this, the project focused on a core set of features that directly supported the system's primary goal, which was to assist users in managing activities, organising schedule information, and making better use of fragmented free time through practical recommendations.

The enhancements that were included in the project's scope were chosen since they directly improved the application's basic functionality. Task management was the system's fundamental component. Through the UI, users can create tasks, inspect them and delete them when no longer needed. Additionally, work could be marked as either pending or completed thanks to the application's support for task status updates. This feature made the system more helpful than a straightforward static list by providing users with a clearer picture of completed and ongoing tasks.

The second major area within scope was the management of timetables. In addition to creating timetable entries, users could view planned sessions and remove items as needed. This feature of the program was essential to the GapMap concept as a whole since the timetable data offered the framework needed to find open spaces

between planned activities. The application could not have provided relevant time-based recommendations without this capability.

Data quality and validation were also crucial components of the project's scope. To make sure that erroneous data could not just enter the system, deadline validation was included on both the client and server sides. For instance, deadlines that were earlier than the present date were avoided, which helped to preserve the precision and use of task data that was recorded. It was a conscious design decision to incorporate validation at several levels since this enhanced usability and reliability by providing the user with instant feedback and shielding the backend from inconsistent or nonsensical input.

One of the most distinctive features included in scope was the recommendation functionality. In order to produce work recommendations, the system was built to identify significant open intervals between scheduled sessions. This was the project's primary innovation since it advanced the application beyond simple information retrieval and storage. Additionally, a user-defined active day window was included so that suggestions could only be produced during the hours that the user deemed pertinent. This stopped the algorithm from perceiving every unscheduled hour of the day as equally accessible for work, which made the suggestion logic more flexible and realistic.

The project's scope also includes employing SQLite for persistent data storage and integrating the front-end and back-end via an API layer. These components were crucial since they ensured the program operated as a true full-stack system instead of a disconnected prototype. The database provided persistent storage so that tasks and timetable entries were accessible between sessions; the front-end handled the presentation of information and receiving user input; the back-end handled logic and validation. Dashboard summaries were also included to the scope in order to provide the user with important information in a format that is easier to understand and use. This made crucial information simpler to quickly understand, which enhanced the application's overall usefulness.

Last but not least, manual testing and iterative refinement were important parts of the project's scope. Instead of seeing testing as a small, last-minute task, it was seen as an important part of the development process. The system was tested over and over again, bugs were found and fixed, and usability problems were worked out over time. This made sure that the final product was not only fully functional, but also more stable and better for real users.

At the same time, the project purposefully left out a number of features. There was no user authentication or account management, so the app was made for one person to use instead of a group of people with secure personal accounts. The system also did not have any collaboration features, so users couldn't share task lists, schedules, or talk to each other. These features could be useful in a bigger business system, but they weren't necessary to show how GapMap works.

Other features that were left out were notifications and reminder emails, third-party calendar syncing, and advanced analytics or predictive features. These were left out because they would have added more technical complexity and outside dependencies that were not needed for the project to be successful. Recommendations based on machine learning were also left out. Even though this method might have allowed for more advanced personalisation, it wasn't needed to reach the project's main goals, and a rule-based recommendation method was better suited to the level and time frame of the work.

Also, the final user interface did not show a fully developed edit workflow for each entity, and it didn't include native mobile deployment either. These choices helped keep a realistic balance between being ambitious and finishing things. The goal of the project was to show off strong full-stack development, good design choices, and a set of useful features that all worked together, not to try to add every possible extension.

In summary, GapMap was a suitable project to do in second year as it presented a challenging task in terms of software engineering while not being overly ambitious. The project had a well-structured frontend, backend, and database; it contained recommendation systems and validation; it also tackled a practical problem. By carefully scoping out the project, it remained on track and relevant throughout its duration.

1.5 Report Overview

The report begins with a discussion about the background, problem statement, objectives, and scope of GapMap. Following that, an evaluation is conducted about the planning, research, and requirement gathering carried out for the design. Thereafter, an analysis of the prototyping process along with iterations is discussed, giving a detailed insight into the development process of GapMap. Further in the report, an overview is provided about the finalized design specifications along with technical development of the frontend, backend, database, and validation. This is followed by an entire testing process chapter, after which a critique is made on the application.

2 PLANNING, RESEARCH AND REQUIREMENTS

2.1 Initial Research

GapMap's initial research period focused on having an accurate understanding of the user problem at hand as well as the different types of software currently found in the overall productivity environment. This research was critical since it ensured that the design being developed did not merely focus on repeating what was typical of normal software but rather addressed an actual gap in the methods being utilized. The research therefore included an assessment of current software as well as the ability for such software to meet the unique challenge that GapMap seeks to solve through the use of fragmented periods of time.

At this point, we looked at a number of different types of software. These included digital calendar systems, generic to-do list apps, study planning tools, and larger productivity platforms that try to combine task tracking with scheduling. Each group gave useful information about common design patterns, their pros and cons. By looking at these different kinds of systems, it was possible to figure out which features were already well supported in the market, and which parts of the user problem still needed more attention.

The investigation found that many productivity tools that are already out there work well in one area but not so well in another. For example, calendar systems are great at showing structured schedules and letting users know when they are busy or free. They give you a good visual overview of your time commitments and are often good for planning around appointments. They don't usually help the user figure out which piece of work would be the best choice for a short free window, though. A calendar might show that there is a 30-minute break between two sessions, but it usually doesn't say what that time should be used for, like studying, reading, managing email, or making progress on an assignment.

Traditional task management systems have the opposite strengths and weaknesses. Most of the time, these apps are good at storing, organising, and prioritising tasks. They also often have options for due dates, labels, status indicators, and sorting. This helps people keep track of what needs to be done, but most of the time they expect the user to decide when to finish each task. In other words, they are good at storing tasks, but they don't really connect tasks to the user's daily schedule. This makes it hard to go from knowing what needs to be done to deciding what can be done in the time available.

Study planner apps were also looked at as part of the research. These systems are often better for academic users and may have features for organising modules, planning revisions, or keeping track of assignments. But the research showed that they don't often link unfinished work directly to real gaps in the schedule in a way

that is both dynamic and useful. A lot of them still use static planning instead of suggestions that take the situation into account. So, while they might help people keep track of their subjects and due dates, they don't always help them decide what to do with a short free period.

These results helped show that GapMap had a clear and useful goal. There are a lot of useful tools in the surrounding productivity space, but not many systems are made to connect the timetable structure with task selection for short periods of time. This information backed up the main idea of the project and made the choice to make a focused application instead of a general-purpose productivity platform even stronger.

The research phase also had a big impact on how the system looked and how people interacted with it. One of the most important things we learned from looking at existing applications is that productivity software can get very messy very quickly when it tries to fit too many features on one screen. A lot of platforms try to offer calendars, task lists, reminders, notes, analytics, and communication tools all at once. This can make the interface too busy and hard to use. This is especially bad for systems that are designed for students, where users often need quick access to information instead of a lot of menus and controls.

In response, GapMap was purposely built with a few focused pages instead of one dashboard with too much information. This design choice was made to make things clearer, lighten the cognitive load, and make navigation easier. The project didn't try to make a big, all-in-one life management platform. Instead, it focused on a smaller, more unique goal. Instead of trying to improve every part of productivity at the expense of usability, the goal was to make a system that did a specific job well.

In addition to looking at existing applications, the research phase also looked at more general user-centred design principles. We used ideas like consistency, being able to see the status of the system, getting useful feedback, stopping errors, and making action labels clear to help us make decisions about the interface later. These ideas had a big impact on how forms, navigation, and interactive controls were made. For instance, clear labels and consistent layouts make it easier for users to know where to add tasks or schedule entries. Feedback messages and validation support help keep things from getting confusing and make sure users don't enter the wrong information. Preventing mistakes was especially important in places like deadline entry, where bad data could make the system less reliable.

Thinking about these rules early on in the project helped make sure that design choices were made based on how easy they were to use, not how new they were. This was a key part of the development process because the goal was not just to make the interface look good, but also to make a system that users could quickly understand and use with confidence. The research thus informed both the conceptual framework of GapMap and the practical design choices that affected the ultimate execution.

2.2 User and Stakeholder Considerations

The primary target users of GapMap are university students. We chose this group because the way they work and manage their time is very similar to the problem the system is meant to solve. Their weekly schedules are often made up of broken-up and irregular blocks of time for things like lectures, lab work, tutorials, private study, preparing for coursework, part-time jobs, and personal obligations. Because of this, students often have a mix of short and long free times throughout the day. At the same time, they often have to do several things at once, each with its own deadline, level of urgency, and level of effort. This makes it hard to remember what needs to be done and to figure out what can realistically be done in a certain amount of time.

An effective system for these users needs to do more than just store data. It should make it easy and quick to add tasks and schedule sessions without having to do a lot of work. The app is less useful and less likely to be used regularly if a student has to go through too many screens, fill out too many fields, or figure out a messy interface. Because of this, GapMap was made to make it easy to enter tasks, see deadlines clearly, and get to the right features right away. The goal was to make the system easy to use in short bursts of time, like when you need to do something quickly.

Reliability is another important need for the main user group. If the data that the system is based on is wrong or doesn't match up, students are not likely to trust the recommendations or summaries it makes. Because of this, the app had to have a way to stop bad input, especially for things like deadlines and schedule entries. It's important to stop users from entering wrong data not only for technical reasons, but also for user-centred reasons, because it helps keep faith in the system's usefulness. The recommendations that come from the information in the app are more likely to be seen as useful and worth following if the information is correct.

It is also assumed that the person who will use GapMap will value clarity over unnecessary complexity. Visual appeal is still important, but a productivity tool only works if users can quickly understand it and feel sure about what they're doing. An interface that is very decorative or has a lot of features may look great, but it can make things harder to use if it is confusing, inconsistent, or takes a long time to understand. This understanding made the system's design favour simplicity and ease of reading. Forms were meant to be straightforward and simple to fill out, cards and summaries were meant to be easy to read at a glance, page names were meant to be clear and descriptive, and actions were meant to be as clear as possible. These choices were meant to help the user understand things quickly and make things easier for their brain.

The main audience for GapMap is university students, but the idea behind it may also be useful for other groups of users. For instance, professionals who are just starting out or people who have meetings on a regular basis may have similar problems when trying to use short breaks between fixed commitments. But these

groups weren't the main focus during development. Instead, the design choices were mostly based on the habits, needs, and limitations of student life.

There are also secondary stakeholders who are involved in the project besides the end users. These are people like tutors, assessors, and other reviewers who see the application as a technical artefact instead of a tool for getting things done every day. For these stakeholders, the system's value is not only in what it does, but also in how well it was made and put into place. They are more likely to care about how the system is set up, how well the full-stack integration works, whether the technologies used are right, and how easy it is to maintain and add new features to the codebase. This means that the project had to meet both practical user needs and academic or technical standards.

People who might use the product in the future can also be seen as stakeholders in a wider sense. They are not directly involved in the current project, but they show how the system could grow in the future. In the future, GapMap might have more features, like managing accounts, working together, making edits easier, or connecting to calendars. Thinking about these possible future users helped us decide to build the project in a way that is structured and can be expanded, even though not all of the advanced features are included in the current scope.

2.3 Functional Requirements

The functional requirements of the system spell out the exact tasks and actions that GapMap was supposed to do to help the project reach its main goals. The project aim tells you what the application is supposed to do in general, while the functional requirements spell out what the system can actually do. GapMap's requirements included making it easy for users to manage tasks, organise schedule data, get helpful suggestions, and interact with information in a reliable and structured way.

A major area of functional requirement concerned task management. Users should be able to add a task, see all of their saved tasks, change the status of a task from pending to completed, delete a task, and edit an existing task. These features were very important because storing tasks and keeping track of progress were two of the main parts of the app. The user needed to be able to write down their work, look it over later, and change it as their situation changed. The system also needed to keep track of deadlines and reject invalid values so that tasks stayed useful and correct. This validation requirement was especially important because if the deadline data wasn't reliable, the whole application wouldn't be as useful, and any recommendations made from it would be less helpful.

A second set of functional requirements had to do with managing schedules. The program had to be able to add new timetable entries, show existing ones clearly, change existing ones, and delete them when they were no longer needed. It also needed to check that the day and time entered for timetable entries were correct so

that wrong or illogical timetable data wouldn't be saved. These requirements were necessary because the timetable part wasn't just an optional extra; it was a key part of how the system made recommendations. The application would not be able to reliably find free periods if it couldn't store timetable data correctly and show it clearly.

The recommendation feature added more functional requirements that set GapMap apart from a regular task manager. The system had to use both task and timetable data to make recommendations, find free time between timetable sessions, and let the user set an active day window. This meant that the app couldn't just store data. It had to process that information in a way that helped people make decisions. It was especially important to have an active day window because it let the user choose which parts of the day should be open to recommendations. This made the system more realistic by recognising that not all free time is really usable.

The system needed to show the best recommendation along with other options, as well as find good work for free time. This was a helpful requirement because decisions about productivity are rarely clear-cut. A user might want to pick from a few reasonable tasks based on how much energy they have, how urgent the task is, or what they want to do. Giving them a choice of tasks is more helpful and flexible than just giving them one set suggestion. The system also needed to show summary statistics on a dashboard so that users could quickly see all of the work and schedule-related information they had stored. This requirement made the site easier to use by making important information easy to find instead of hiding it on separate pages.

Lastly, the system had to keep all of its data in a database for a long time. This was a key technical requirement because a productivity app wouldn't be very useful if tasks and schedule entries disappeared after each session. Persistent data storage made sure that GapMap could be used continuously and proved that it was a real full-stack application, not just a temporary prototype.

The planned functional requirements for the project were therefore as follows:

- add a task
- display all saved tasks
- edit an existing task
- delete a task
- change task status between pending and completed
- store deadline data and reject invalid values
- add a timetable entry
- display timetable entries clearly

- edit an existing timetable entry
- delete a timetable entry
- validate day and time input for timetable entries
- generate recommendations using task and timetable data
- detect free gaps between timetable sessions
- allow the user to define an active day window
- show the best recommendation and alternative options
- display summary statistics on a dashboard
- store all data persistently in a database

The system does a good job of meeting most of these planned requirements for the current minimum viable product. The final build fully supports making tasks, showing tasks, deleting tasks, updating task statuses, making timetables, showing timetables, deleting timetables, making recommendations, letting users choose a day-window, and showing dashboard summaries. All of these features work together to make a functional and coherent system that solves the main problem that was found at the start of the project.

The original planned requirements included editing functionality, but in the current version, it is only partially available through the final interface. Backend update logic has been created, which lays the groundwork for future expansion. However, the user-facing system does not yet have a fully polished edit workflow for all entities. This means that the application has a clear path for further development, but the current MVP focuses on the most important core features that are needed to make the system work, be usable, and be technically complete within the project's time frame.

2.4 Non-Functional Requirements

The functional requirements tell GapMap what it needs to do, while the non-functional requirements tell it how well it should do those things. They talk about how the app should work in terms of usability, reliability, maintainability, performance, and responsiveness. These requirements are important because a system may have all the right features on paper, but if it's confusing, unreliable, hard to maintain, or slow to use, it won't give users a good experience. These features are especially important for a productivity app like GapMap because the system is meant to help people make decisions quickly and use it every day.

Since the application is aimed primarily at university students, the interface needed to be simple, readable and easy to navigate. Users should be able to quickly figure

out what each page is for and switch between main functions without getting confused. It was important that actions like adding a task, deleting an entry, or viewing recommendations were clearly labelled so that the interface stayed easy to understand. Forms were also supposed to give users immediate feedback whenever possible. This helps users fix mistakes quickly and reduces uncertainty. Also, the recommendations page had to be easy to understand without any special help. This was very important because the recommendation feature is one of the most unique parts of the system. If users couldn't easily understand why certain suggestions were being made, the feature would lose value.

A second important category was reliability. The system needed to behave consistently and protect the quality of stored data. If at all possible, invalid input should be stopped or rejected, especially when it comes to deadlines that are earlier than the current date or wrong timetable values. Data that was stored also had to stay the same after a page refresh or application restart. A productivity system wouldn't be useful if it was easy to lose or corrupt information. It was also expected that the recommendation logic would act the same way for the same inputs. This requirement is important because users are more likely to trust the app if they know what to expect from it. Also, when there are mistakes, the system should show readable messages instead of failing without saying anything. Clear error messages help both users and developers understand what went wrong and build trust.

The project also required a strong level of maintainability. GapMap was a full-stack application, so the code had to be organised in a way that made it easy to read, maintain, and add new features in the future. Because of this, the codebase should be broken up into logical layers so that different tasks are kept separate. Frontend and backend issues had to be clearly separated so that there was less confusion and it was easier to make changes. Model files were meant to centralise access to the database, which makes things more organised and cuts down on the need to repeat logic throughout the system. It was also important that the validation rules were easy to understand and could be used in different parts of the application in the same way. These requirements for maintainability were important not just for the current project, but also for showing good development practice and making it easier to make improvements in the future.

Performance was another important area that wasn't functional. GapMap is meant to help people be productive every day, so it had to be able to quickly load task lists, submit forms, and make recommendations. Long waits would make the app less useful and get in the way of the user's work. The system also had to be light enough to run on regular student computers without needing a lot of processing power or memory. This was an important thing to think about because the people who will use the system will probably do so on regular laptops instead of specialised devices.

Lastly, responsiveness was a key part of the non-functional requirements. The app had to be easy to read and use on screens of different sizes, like those on laptops

and tablets. Content should not be hard to read on smaller screens, and big chunks of information should not look weird when they are stretched across wide displays. Buttons, forms, and other interactive controls also had to be easy to use no matter how big the screen was. These requirements helped make sure that the system would work well and be easy to use on the devices that the target audience was most likely to use.

The planned non-functional requirements for the project were therefore as follows.

2.4.1 Usability requirements

- the interface should be simple, readable and easy to navigate
- actions should be clearly labelled
- forms should provide immediate feedback where possible
- the recommendations page should be understandable without specialist explanation

2.4.2 Reliability requirements

- invalid input should be prevented or rejected
- stored data should remain consistent after refresh or restart
- recommendation logic should behave consistently for the same inputs
- errors should produce readable messages rather than silent failure

2.4.3 Maintainability requirements

- code should be separated into sensible layers
- frontend and backend concerns should be clearly distinguished
- database access should be centralised through model files
- validation rules should be understandable and reusable

2.4.4 Performance requirements

- common actions such as loading lists, submitting forms and generating recommendations should respond promptly
- the application should remain lightweight enough for normal student hardware

2.4.5 Responsiveness requirements

- content should remain readable across common laptop and tablet widths
- large content blocks should not stretch awkwardly across wide screens
- buttons and forms should remain easy to interact with

2.5 Project Planning

The project was planned in stages and with dependencies in mind so that development could move forward in a logical and manageable way. The project didn't start with the visual interface; it started with the backend structure and the data model. This choice was made because the way the app worked depended a lot on how tasks, timetable entries, and the recommendation logic that was built on top of them interacted with each other. Setting these foundations first lowered the chance of having to redesign later and made it easier to make sure that the system architecture supported the features that were planned.

This planning method was important because GapMap isn't just a static interface. The main value of this service comes from how the stored task data and timetable data work together to make useful suggestions. Because of this, it was important to figure out how to show, check, and get information before focusing on presentation. Building the project in layers meant that each stage could support the next, which made the development process more reliable and less likely to have major structural problems later on.

The overall plan can be broken down into a series of milestones, each of which adds a specific part to the final system.

2.5.1 Stage 1: define problem, narrow scope and describe MVP

The first step of the project was to make sure that the problem the system was supposed to solve was clear and that the project stayed realistic given the time frame. This meant figuring out what the main user need was, looking at the productivity space around it, and deciding which features were necessary for a minimum viable product. At this point, the project was intentionally made smaller so that the final system would stay focused and be possible to build. This was a big step forward because it set the limits on the work and kept the project from growing into too many features to handle.

2.5.2 Stage 2: design the database schema and backend skeleton

After defining the main problem and the scope, the next step was to build the application's structural base. This involved making the database schema to show tasks, timetable entries, and the relationships needed for recommendation logic later on. At the same time, the backend skeleton was set up so that routes, models, and the main server-side structure were all in place before adding more specific features. By starting with the database and backend, we made sure that the frontend work

would connect to a stable and well-organised system instead of relying on temporary guesses.

2.5.3 Stage 3: implement timetable and task CRUD foundations

The third stage was all about making the basic create, read, update, and delete functions for the two main types of data in the system: tasks and timetable entries. These features were the most important parts of the app because the recommendation engine couldn't work until users could add and manage this information correctly. At this point, the main goal was to get the system up and running so that it could store, retrieve, and delete important data. This was also the time to start setting up rules for checking and handling data.

2.5.4 Stage 4: build the recommendation engine

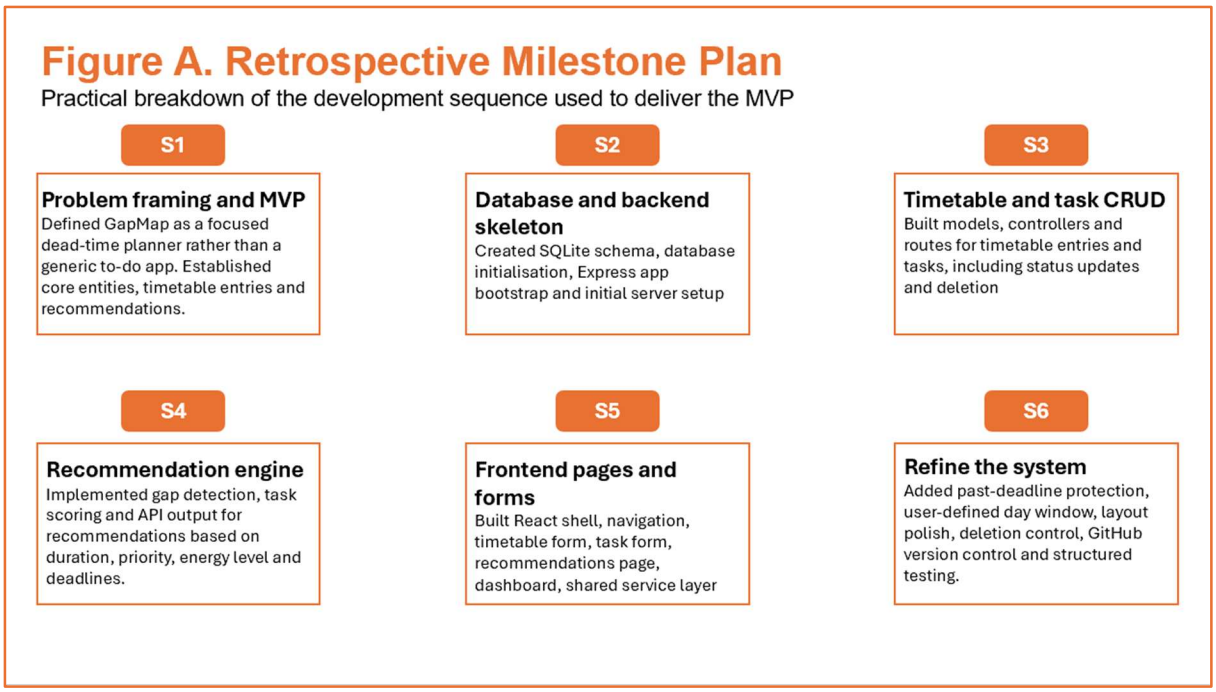
The next big step after getting the core data features to work was to set up the recommendation engine. This stage was very important because it added the feature that set GapMap apart from other maps. It was necessary to use logic to find meaningful free gaps between timetable sessions, think about the user's tasks and deadlines, and come up with suggestions that were realistic for the time available. Putting this stage after the CRUD foundations was a smart choice based on dependencies, since the recommendation system needed structured and valid data to work.

2.5.5 Stage 5: connect frontend pages, forms and API calls

After the backend functionality and recommendation logic were set up, the next step was to connect the frontend pages, forms, and API communication. The goal of this stage was to make the system usable from the user's point of view by giving them a clear way to interact with the underlying features. We linked pages to backend routes, forms to actions that let you create and delete things, and we improved the way data moved between the client and server as a whole. At this point, the project went from being a working backend system to a full-stack application.

2.5.6 Stage 6: improve validation, recommendation realism and layout polish

The last step was all about making things better. Once the main features were working, the focus moved to making the system better. This meant making validation stronger, making the recommendations more realistic and useful, and making the layout and presentation of the interface better. These tasks were important because a system that works technically isn't always easy to use. Last-minute changes made sure that the app worked more reliably, looked more consistent, and better supported the desired user experience.



Even though the milestone plan in Figure A gives a clear picture of the planned order of development, the project didn't go in a straight line. Software development often only shows problems when different parts are put together and tested together. The same was true for GapMap. The staged plan was helpful in some ways, but some problems didn't show up until the backend logic, frontend behaviour, and stored data all worked together as one system. Because of this, the development process became more iterative than the original plan for milestones might have suggested.

This adjustment was particularly evident during the implementation of the recommendation engine and the integration of API-driven frontend behaviour. For instance, the recommendation logic first made gaps that lasted all day because the first version of the gap-detection process was too broad when it came to large unscheduled periods. The core logic was working, but the output didn't always make sense from a real user's point of view. This needed to be improved so that the system could find more useful gaps and make suggestions that were more in line with real job opportunities. Task fetching also failed for a short time after task creation worked because a controller tried to access request data in an unsafe way during GET operations. It wasn't until the creation and retrieval workflows were tested together through the interface that this problem became clear. This kind of problem shows why full-stack development doesn't usually happen in a simple, step-by-step way.

Because of this, the actual development process turned into a cycle of implementation, testing, diagnosis, and improvement. It was common to build features in a basic form, test them when they were connected to other parts of the system, and then make them better when they didn't work as expected or had bugs. This iterative pattern was important because it helped the project go from just finishing basic features to getting a final result that was more reliable and easier to

use. Instead of sticking to the original plan, development stayed flexible enough to deal with problems as they came up. This was especially important for a project like GapMap, where the system's usefulness depends on more than just whether features are there; it also depends on whether they work well together in real life.

The new way of developing things was a balance between structured planning and practical adaptation. The milestone plan was still useful because it set the right course for the build and made sure that the most important dependencies were taken care of in the right order. But the actual implementation process showed that successful development also requires going back to earlier decisions, fixing assumptions, and making logic better once the system starts to work as a whole. In this way, the project followed an iterative version of the original staged plan instead of a set order.

GitHub was also very helpful in planning and keeping track of the development process. It was mostly used for version control, but it also helped organise the project by making a visible record of how the system changed over time. Meaningful commits helped break the build down into important steps, like creating the schema, writing the backend logic, wiring the interface, and later improving the validation. This made it easier to see how the project was changing and gave a clearer way to look at changes.

Version control also made it safer to try out new ideas and make improvements. GitHub made it easier to separate changes, find out when bugs were introduced, and keep a clearer record of development choices as features were added or changed. This was especially helpful in the later stages of the project, when the rules for validation, the realism of recommendations, and the details of the interface were all being improved. So, the commit history was not only a technical backup, but also a simple planning and monitoring tool that helped the project move forward in small steps.

2.6 Development Methodology

GapMap was made using an iterative method and agile-style work habits. The project didn't officially use a full Scrum framework or a strict Kanban process, but its development still followed a lot of the basic agile principles. These were things like delivering in small steps, testing often, fixing bugs quickly, and making improvements based on what was learned during implementation. This method worked well for the project because the final system had to work with several connected layers, such as data storage, backend logic, API communication, and frontend interaction. All of these layers had to work together without fail.

Instead of trying to build the whole application in one go, the development was done one feature at a time. Each feature was treated like a small working part of the system, so that a small but useful piece of behaviour could be built, tested, and then

added to. This method made it easier to keep track of progress because it made sure that development stayed focused on one set of related requirements at a time. It also helped keep the project from getting too abstract because each finished slice made the system better in a way that could be seen and tested.

Technical dependencies between features had a big impact on this sequencing. For instance, the timetable feature was added before the recommendation system because the recommendation system needed timetable data to find useful free gaps. In the same way, task logic was made before the dashboard because the dashboard needed information about task counts, task status, and deadlines. This meant that the order in which things were made was not random. The features were added in an order that showed how the system really worked, starting with the most basic ones so that later parts could build on them.

This project was a good fit for an iterative methodology because it lowered the risk of development. There was a working baseline that could be tested and reviewed at every stage. This made it easier to figure out if a new problem was caused by the most recent change instead of having to look into a lot of changes that happened at the same time. This kind of controlled progression is especially useful in full-stack development, where a bug could come from the frontend, backend, API communication, or database behaviour. It makes it easier to find bugs and follow the trail of code.

The iterative approach also made it possible to make improvements when problems that only showed up when features were connected. Some parts of the system worked differently when they were alone than when they were part of the whole workflow. This is why development naturally turned into a cycle of putting things into action, testing them, figuring out what went wrong, and making them better. A feature could be built in a simple way at first, and then changed when problems with usability, logic, or unexpected interactions became clear. This showed an agile mindset in action, even without formal ceremonies, because the project kept changing based on what was learned during development instead of sticking to a plan that couldn't be changed.

GitHub's version control made this method even stronger throughout the project. Git was not just used to send in the final version; it was also used during the development process. Regular and meaningful commits were used to keep track of progress, keep stable versions of working code, and make it easy to go back to a point in time when later changes caused problems. This made it safer to try new things because developers could be surer that earlier working versions would still be available if they needed them.

GitHub also made the project's overall structure better by making it easier to see how the development was going. Using commit history, it was easier to see and review different stages of the build, like creating the schema, adding backend logic, integrating the interface, and making validation better. This was useful for both

development and project management because it made a record of how the system changed over time. Version control worked as a protection against losing work and as a useful tool for iterative planning and controlled refinement.

3 PROTOTYPING AND ITERATION

3.1 Early Concepts

The initial idea for GapMap was mostly based on the problem it was meant to solve, not on a set visual design or a detailed plan for the interface. The project was based on the idea that many users, especially college students, have short breaks between scheduled sessions that they don't always use well. So, the main idea behind the system was to find this "dead time" and link it to real tasks that need to be done. In this way, GapMap was never meant to be just another way to keep track of tasks. Its unique goal was to connect being aware of your workload with actually using your time wisely.

The idea behind the system was to bring together two types of information that are usually kept separate: what the user needs to do and when they can actually do it. This was the main idea that guided the project from the very beginning. Instead of starting with how the application should look, the design process started by asking what kind of behaviour would make the system really useful. So, the focus was on functionality, usefulness, and decision support, not just looks.

The first design questions were about how things worked and how the system worked. These included figuring out what information the user would need to enter, what would be a meaningful gap between sessions, how the system could suggest a suitable task without making things too complicated, and how the interface could stay simple even though the logic behind it would be more advanced than that of a typical to-do list. These questions were important because they helped figure out what the project was all about. They made sure that the system would stay focused on solving a specific real-world problem instead of turning into a bunch of general productivity features that don't work together.

At this stage of development, the project's idea was clear in terms of its goal, but the interaction design was still not very well thought out. People already had a good idea of what the system should do, but they didn't yet have a full picture of how users would move through the interface or how information would be shown from screen to screen. To put it another way, the definition of the problem was better than the definition of the visual design. This happens a lot in the early stages of software design, especially when a project comes up with a functional idea that is more important than a purely visual one.

During the lower-fidelity prototyping stages, this gap between conceptual clarity and interface clarity was filled. To turn the initial idea into a more useful screen-based design, early sketches and structural planning were used. This helped the project go from an abstract functional idea to a usable application structure. Because of this, the early concept stage was an important part of the later design process. It made

sure that the interface choices made later were still in line with the system's original purpose.

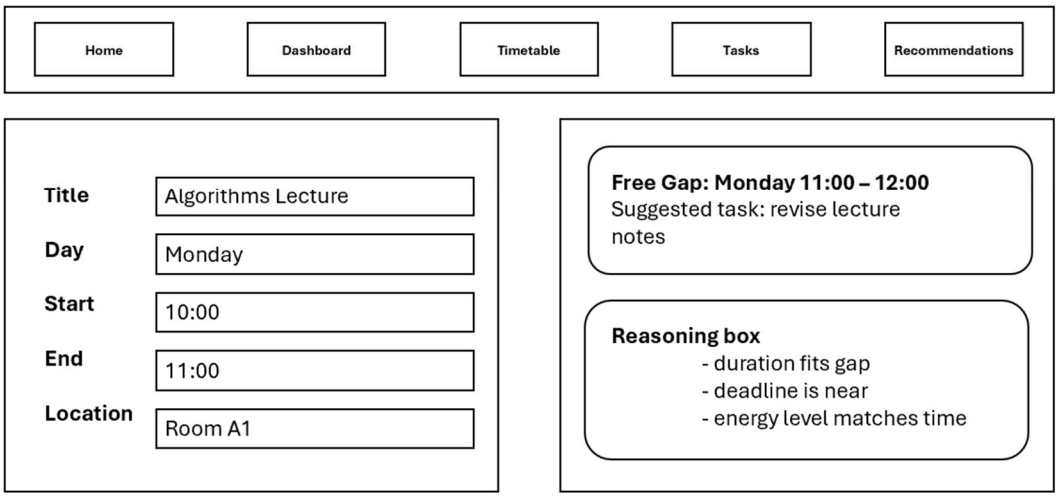
3.2 Low-Fidelity Prototype

The low-fidelity stage was more about structure than making things look good. It used rough layout planning and placeholder screen ideas to figure out what the main pages should be and how the user would move between them.

A reconstructed low-fidelity wireframe is shown below.

Figure B. Reconstructed low-fidelity wireframe

Paper-style concept showing a simple navigation bar, task form and recommendation area



At this point in the development process, the overall structure of the GapMap page was already starting to point to the system's final direction. Even though the interface still wasn't very high quality and didn't look very good, the main functional areas were starting to become clear. There were separate areas for navigation, data entry, and recommendation output, which helped set up the basic flow of how users would interact with the site. This was a key step in the design process because it helped the project go from an idea to a structure that could be used in real life and tested in theory.

At this point, the layout was meant to be simple. The goal of the prototype was not to make the system look finished or impressive, but to make sure that the user could understand how the whole interaction worked. It was especially important to see if the transition from entering tasks and schedule data to getting useful recommendation output made sense and was easy to understand. This was important because GapMap needs a relationship between input and output that is a little more complicated than that of a regular to-do list. If the structure wasn't clear, the app wouldn't be as useful even if the logic behind it was sound.

This part of the planning process also helped figure out which features needed their own space in the system. One important result was the choice to put timetable management and task management on separate pages instead of trying to combine them into one busy interface. This was a good design choice because even though both features are related, they require different kinds of interactions and information. By keeping them apart, there was less visual clutter, better clarity, and the overall navigation model was easier to understand.

The recommendation feature also started to stand out as a separate part of the system instead of being seen as a small extra feature. At this point, it was becoming more and more clear that recommendations needed their own page. This helped GapMap's overall identity because the recommendation system is what sets it apart from other productivity tools the most. Giving this output its own space helped make the point that the app wasn't just for keeping track of tasks and schedules; it was also for turning that information into useful decisions about how to use time.

Another good thing about this stage was that it let us test the flow of information early on without needing to do any advanced styling or frontend work. By concentrating on structure initially, it became feasible to assess the logical coherence of the page and action arrangement prior to dedicating time to visual enhancement. This is a smart design choice because it's usually easier to fix structural problems early on than after a polished interface has already been built on top of them.

But this stage also had some clear limitations. It did a good job of showing the basic structure of the system, but it didn't yet give a strong sense of visual hierarchy, emphasis, or realism. Spacing, typography, colour use, and interactive feedback are all important interface features that were not yet fully developed. The prototype could show that the application made sense in theory, but it couldn't yet show how the system would feel in real life. In other words, it showed that the interface was built correctly, but not yet how good the final user experience would be.

Even though this step was weak, it was very important because it set up the design's structural base. It showed that the system needed separate but connected areas for tasks, timetable data, and recommendations. It also showed that a simpler page-based layout was better than an overloaded all-in-one interface. This gave us a good starting point for later making more detailed wireframes and designing interfaces with higher fidelity.

3.3 Medium-Fidelity Prototype

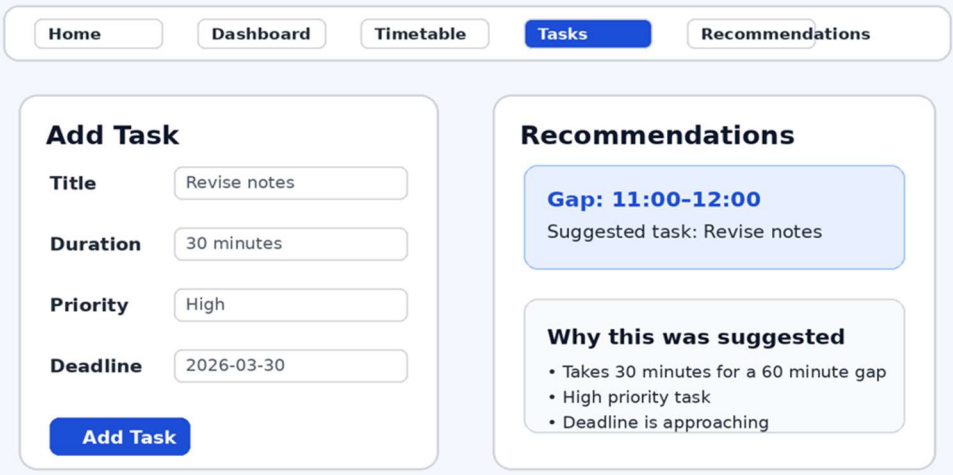
The medium-fidelity stage started when the main pages of GapMap were linked to live data and the placeholder content from before was replaced with working interface parts. At this point, the app had gone past planning its structure and was already in a form that could support meaningful interaction. Through working API

calls, users could add tasks and timetable entries, view stored information, and get results from the backend. You could move between pages, the data lists were showing real content, and shared service logic had been added to make communication between the front end and back end more reliable.

Figure C shows a rebuilt medium-fidelity prototype.

Figure C. Reconstructed medium-fidelity prototype

A cleaner component-based layout with cards, stronger hierarchy and colour-coded action



This step was very important because it was the point at which GapMap went from being just a design idea to a working system. The earlier stages were helpful for figuring out the layout and how to separate features, but the medium-fidelity prototype was the first stage where real user actions could be done and tested correctly. Users could now input data, navigate the interface, and receive genuine system responses instead of hypothetical results. This made the prototype a much better tool for design and evaluation because it showed problems that only show up when a system is used in real life, not just talked about in theory.

One of the best things about this stage was that it made the connection between visual design and system behaviour clearer. The original recommendation logic could be tested in real-world conditions once real timetable data was available. This quickly showed that the first version of the gap-detection process could make gaps that lasted all day, which wasn't realistic. The recommendation feature worked fine, but the output showed a bigger problem with usability and design: just because a system works doesn't mean it gives the user useful results. This was a useful discovery because it showed that GapMap could only be successfully used if the code was correct and the recommendations were based on real opportunities for useful work.

So, the medium-fidelity stage was an important part of the diagnosis. It revealed issues that were not apparent in the lower-fidelity structure, especially those related to realism, system behaviour, and the interpretation of information when presented in context. It was clear that design evaluation and technical testing could not be completely separate because the quality of the user experience depended a lot on how well the underlying logic worked with real data. This made the project's iterative development approach even more valuable. In this approach, features were built, tested, reviewed, and then improved based on how people used them.

The medium-fidelity prototype was clearly better than the low-fidelity one when it came to looks. The interface was easier to read, the layout was more based on components, and the use of cards, grouped sections, and clearer navigation made the system more organised. The rebuilt prototype in Figure C shows how the design was starting to make the hierarchy clearer, with separate areas for entering data and making recommendations. The navigation bar also suggests a more advanced information architecture, with separate pages for Home, Dashboard, Timetable, Tasks, and Recommendations. Also, using colour-coded actions and boxed content areas made things clearer and made it easier to scan the interface.

But even with these changes, the design was still not finished at this point. There were still some places where the layout was a little off, the spacing wasn't always the same, and the visual system didn't feel like it was ready for a polished final submission yet. The interface worked and was easy to read, but it still looked like it was in the process of being finished. Some parts felt stronger than others, and the overall design language wasn't quite there yet to make the app feel like one whole thing. This is typical of medium-fidelity development, where the main goal is to make the system usable and testable before worrying about the final presentation quality.

The medium-fidelity prototype was a big step forward for the project, even though it had some problems. It confirmed that the earlier changes to the page structure and feature separation could work in real life, and it made the recommendation feature stand out more clearly as the app's main focus. It also gave the project a much stronger foundation for improvement, since decisions about how the system should look and work could now be based on how it was actually used instead of on guesses.

3.4 High-Fidelity / Final Prototype

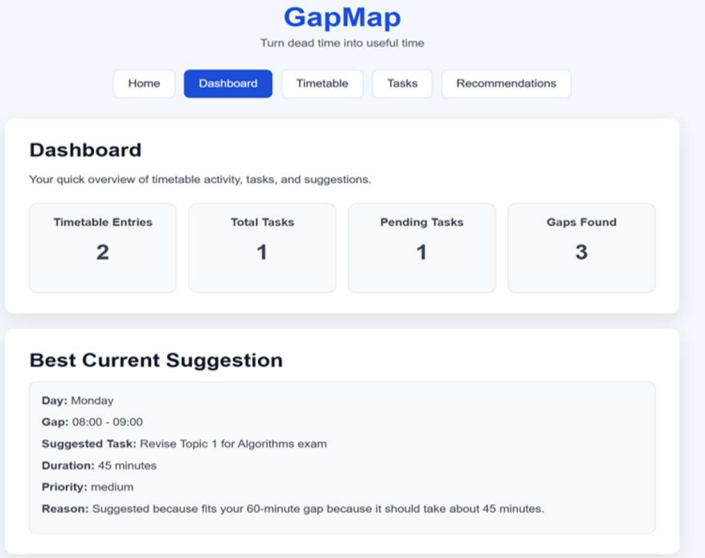
The high-fidelity stage of development is very similar to the final version of GapMap that was delivered. The system had progressed beyond experimentation and structural testing to a stage appropriate for presentation as a comprehensive academic product. The final interface combined the most important things we learned from the first concept, low-fidelity structure, and medium-fidelity

implementation stages. It combined full-stack functionality with a more planned visual design, making an app that was both useful and easy to understand.

Figure D shows an example of what the final build will look like.

Figure D. High-Fidelity / Final prototype

Final dashboard screen showing summary statistics and a current recommendation preview



The final prototype's layout is one of the most noticeable things about it. It is more controlled and consistent. The interface has a centred page structure that makes the app look balanced and focused. This helps the content feel organised instead of scattered. A reusable navigation bar that looks the same throughout the system and has active-state highlighting so users can easily see which section they are in right now supports this. This way of navigating makes it easier to find your way around and makes the whole experience better, especially in a multi-page app where it's important to be able to see where you're going.

The high-fidelity version also has clearer forms, better data cards, and a dashboard that shows summary statistics in a way that is easy to read. These features show that there is a stronger sense of visual hierarchy than there was in the earlier prototypes. Important information is organised into separate sections, headings are more visible, and it's easier for the user to tell the difference between main content and supporting detail. This is especially clear in the dashboard view, where summary cards show all the timetable entries, total tasks, pending work, and detected gaps right away. There is also a separate section that shows the current best recommendation in a more detailed way. This layered presentation makes it easy for people to understand the system without giving them too much information at once.

At this point in the design process, it was important to choose clarity over extra decoration. This was a planned and good choice for a productivity app. Tools that are

meant to help with planning and making decisions should be easy to read, look reliable, and be quick to scan. An interface that is too fancy or complicated to look at may get people's attention, but it can also make it harder to use if it gets in the way of what the user wants to do. So, for GapMap, the focus was on readability, structure, and trustworthiness instead of just making things look new. The final prototype shows this by using a simple and professional visual style that fits with the app's academic purpose.

The interface uses colour on purpose and in a limited way, rather than as a constant decorative element. In the final build, stronger colour emphasis is only used in places where it really helps, like active navigation states, important action buttons, danger actions, and components related to recommendations. This careful use of colour helps guide the user's attention without making the interface feel too busy. It also makes things more consistent by making the visual language clearer, with colour used to show state or emphasis instead of being used randomly.

The high-fidelity version is much easier to use than the earlier prototypes because it makes it clearer what each part of the system does. Users can easily tell the difference between data entry fields, list-based views, dashboard summaries, and recommendation outputs. This is a big deal because GapMap has a lot of connected features that could easily get mixed up if they aren't clearly separated. The final prototype lowers this risk by making it clearer what each page is for and by organising related information into predictable, well-structured parts.

The final build also strikes a better balance between how well it works and how easy it is to use. Earlier stages had already shown that the system could do useful things like store tasks, manage schedule entries, and make suggestions. The high-fidelity stage's job was to show off these features in a way that made sense, was useful, and felt like they could be used in real life. In this way, the final prototype is not only better looking than the earlier versions, but it also makes the system easier to understand, which makes it a better productivity tool.

Another beneficial aspect about the high-fidelity stage is that it makes GapMap seem more like a focused app and less like a generic to-do list. The presence of dashboard summaries, recommendation output, and clearly defined functional areas makes it clear that the app is designed to solve the specific problem of turning fragmented free time into useful work opportunities. This makes the final product feel more unique and makes the recommendation feature feel more important instead of secondary.

3.5 Iterative Changes

One of the best things about the GapMap project was how much better both design and technical choices got when real problems came up during development. Instead of treating the first version as final, the system was tested and improved many times

as problems came up during use. This process was important because a lot of the biggest problems didn't show up until the application was used from start to finish with real data and realistic workflows. The changes made the project more reliable, better at making recommendations, easier to use, and better at balancing the visuals. They also show that the project was built through evidence-based improvement rather than one-time construction.

A clear example of this iterative improvement process occurred within the task workflow. At one stage, task creation appeared to succeed, but the task list failed to refresh immediately afterwards. This made things confusing and inconsistent for the user: the interface said it worked, but then the page failed to show the updated data. The investigation found that the problem was caused by the task controller trying to access request data in an unsafe way while GET operations were going on. As a result, the helper logic that gets the user_id was changed to use optional chaining, which makes it safer to handle request values. This change was needed because a successful action should never be followed by a refresh failure that could have been avoided. After the controller was fixed, users could reliably create and reload tasks, which made the workflow feel stable and trustworthy.

A second and more conceptually important issue emerged in the recommendation system. Early versions of the application produced very large start-of-day and end-of-day gaps, which led to unrealistic recommendations for windows such as 11:00 to 20:00. Even though the logic was technically creating available periods, the suggestions that came out of it didn't show realistic chances for focused work, which made the feature less credible. This problem had a big impact on how useful the system was because the recommendation engine is what makes GapMap what it is. To fix the problem, the gap-detection logic was rewritten so that it only found real gaps between real timetable sessions. Also, an active day window that the user can set up was added so that sessions could be clipped to the right working period without making fake edge gaps. This improvement was necessary because the quality of the recommendations depends on how realistic the time windows being looked at are. Following these changes, the system began to produce suggestions that were far more believable and much better aligned with the original aim of helping users use fragmented time productively.

There was another problem with usability on the recommendations page itself. After each page refresh, the user-selected active day window would reset. This meant that the user had to enter their preferred start and end times every time they went back to the page. Even though this didn't stop the system from working, it made the user experience less smooth. Productivity software should cut down on the need for repeated work as much as possible, especially for settings that are likely to stay the same between sessions. To fix this, the chosen day-window values were saved in localStorage and automatically loaded when the page was opened. This change made the recommendations page more stateful and easier to use. It also improved continuity and cut down on the need to configure things over and over again.

Validation also became a key area for improvement. At one point, users could enter task deadlines that were earlier than the current date. From a planning point of view, this was not a good idea because past deadlines don't help with active forward-looking task management and can make people less sure about the stored data. The problem showed that the system as a whole needed better security, not just in the user interface. Because of this, validation was added in two places. On the front end, the form logic and date input were changed so that past dates couldn't be submitted. The controller on the back end was also changed so that it would not accept past deadlines even if the front-end protections were not working. This two-layer method was a big step forward because validation should protect both the data that is stored and the experience of the user right away. The result was a stronger and more dependable system that always stops invalid deadlines.

Not all improvements were just technical. Some changes were made to make the interface look better and easier to read. One of these problems was that the main content area looked too wide and too left-aligned on bigger screens. This made parts of the application, like the dashboard, look unbalanced and less professional. In response, the rules for the layout were changed so that the main content area could only be as wide as the centre. This change made the page more balanced, easier to read, and gave the system a more professional look overall. This improvement was fairly simple, but it made a big difference in how complete and well-designed the final interface looked.

These examples show that GapMap's refinement wasn't just about making small visual changes; it was a key part of the development process. There was clear evidence from real use for each problem that was found, and each change was made for a clear reason related to usability, reliability, or realism. The changes that came about were useful and could be measured. For example, task workflows became more stable, recommendation output became more believable, user preferences became permanent, invalid data was blocked more effectively, and the interface became easier to read.

4 DESIGN SPECIFICATION

4.1 Final System Overview

The final version of GapMap is a full-stack web app that helps users better manage their workloads by putting task management, timetable organization, recommendation generation, and summary reporting all in one place. The finished application takes the main ideas that were set out earlier in the project and turns them into a working product that is both technically sound and easy for the user to understand. Instead of just being a list of things to do, the system creates a more

integrated productivity environment where stored tasks and stored timetable sessions can work together to make useful suggestions for free time.

The final system lets users enter and manage two main types of information: tasks and timetable sessions. You can store tasks with useful metadata that helps you plan and set priorities. Timetable entries give the system the structured schedule it needs to find free time between commitments. Once this information is available, the recommendation feature looks for empty spaces in the schedule and suggests tasks that can be done during those times. The app also has a dashboard that shows summary information so that users can quickly see how their stored data is doing and what the current recommendation context is.

From the user's perspective, the finished system delivers several important capabilities. Users are able to add tasks with meaningful supporting information, such as duration, priority and deadline, allowing work to be stored in a more useful way than a basic text list. They can also add timetable sessions to the app so that their day has a clear structure. The system then lets them see work that is still open and work that has been finished, which helps them keep track of progress as tasks move through different states. Users can also see where there are free spaces between stored sessions and get suggestions for tasks that would fit in those spaces. A user-defined active day window also affects the recommendations. This lets the user decide which hours are relevant for generating suggestions. Lastly, the dashboard shows a high-level summary of the information that has been stored and the current state of the recommendations. This makes it easier to understand the system at a glance.

The main user-facing capabilities of the final application can therefore be summarised as follows:

- adding tasks with meaningful metadata
- adding timetable sessions
- viewing pending and completed work
- seeing where free gaps exist between sessions
- receiving a recommended task for each gap
- choosing an active day window for recommendation generation
- viewing a high-level summary of stored information and current recommendation state

To support these capabilities, the application is organised into five main pages:

- Home
- Dashboard

- Timetable
- Tasks
- Recommendations

During development, the page structure was carefully planned out. The system doesn't put all of its features into one overloaded interface. Instead, it groups them by clear user goals. The Home page gives you a first look at the app and helps you get started. The Dashboard gives the user a short overview of the data that is stored and the status of the recommendations. The Timetable page is where you make and show scheduled sessions. The Tasks page is all about managing work items, including their status and other important information. The Recommendations page is the most important part of the system because it shows tasks that can be done during identified gaps in the timetable. This structure makes the final build easier to use, understand, and evaluate as a whole software product.

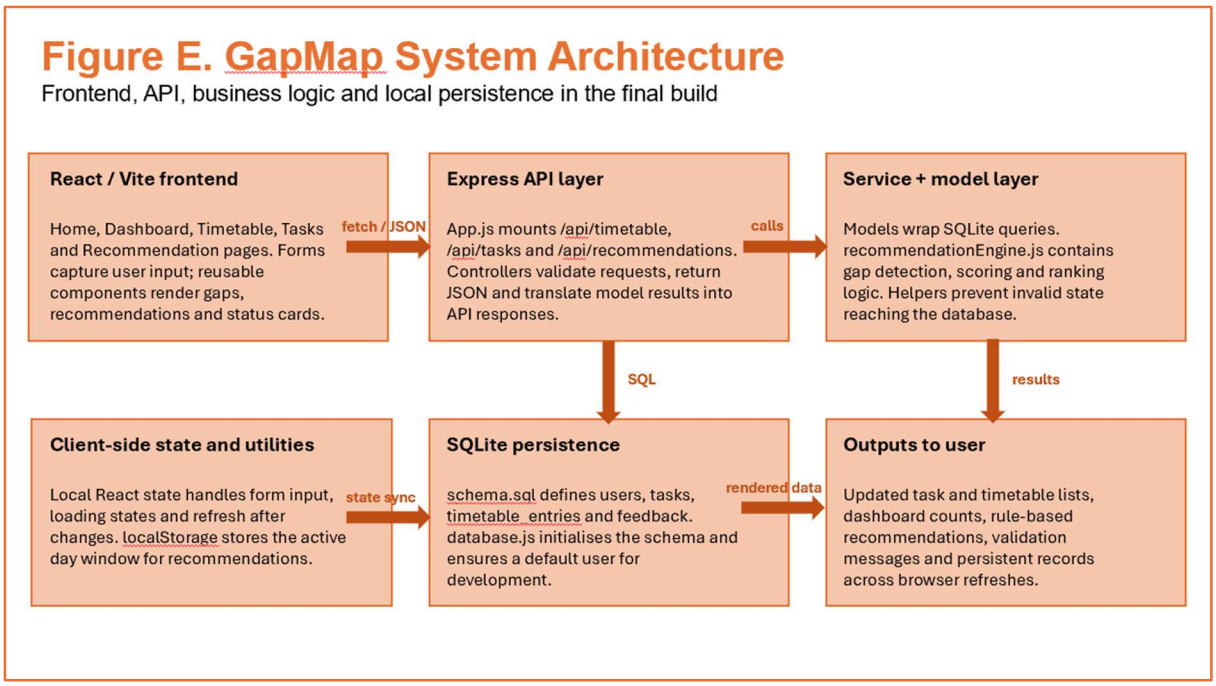
From both a user and an academic point of view, this separation of pages also makes the whole system easier to understand. Everyday users benefit because it's easier to follow the workflow and each page has a clear purpose. For instance, users know where to go to add tasks, look over their schedule, and get ideas for how to use their time. The structure also makes it easier to evaluate the project because the main parts of the system are clearly visible and logically separated. This shows that the app was made to work as a full-stack solution instead of just a bunch of screens that don't connect to each other.

The final system also shows that the project's original goal was to put usefulness ahead of unnecessary complexity. Each page helps the user understand what they need to do, when they have free time, and what work is best for the time they have. The dashboard shows an overview, the timetable and task pages let you enter and manage data, and the recommendations page turns stored information into useful advice. When put together, these parts make a system that is focused, easy to understand, and in line with the original problem the project was supposed to solve.

4.2 System Architecture

The frontend, backend, service layer, and database of GapMap are all clearly separated, which is how a traditional full-stack architecture works. We chose this architectural style because it fits the project's size and goals. It is complicated enough to show real full-stack development, but not so complicated that a second-year computer science project can't understand, maintain, or use it.

Figure E shows how the whole system is put together.



The application is made with React and Vite on the front end. React was used to make a modular user interface with separate pages and reusable parts. Vite, on the other hand, was used to create a lightweight and efficient development environment. The frontend is in charge of displaying the user interface, taking user input, and showing returned data in a way that is easy to read and understand. In GapMap, this includes the Home, Dashboard, Timetable, Tasks, and Recommendations pages, as well as reusable interface parts like cards, forms, navigation, and list-based displays. This layer is mostly about interaction and presentation, not business logic. This helps keep the client side focused and easier to manage.

The fetch-based API layer defined in api.js lets the front end and back end talk to each other. This gives the server a central place to handle requests, which makes the application more consistent and cuts down on duplication. Instead of putting request logic in different places on each page, API calls are put in one place and then used again when needed. This design makes it easier to keep the app up to date because it will be easier to make changes to endpoints or how requests are handled in the future.

Node.js and Express are used to build the backend. This layer is in charge of how the application works. It gets requests from the front end, checks the data that comes in, sends work to the right business logic or model functions, and sends back structured JSON responses. The backend is set up in a way that separates responsibilities into logical layers, with routes, controllers, and models. Routes tell you what API endpoints are available, controllers handle how requests and responses work, and model files handle how to talk to the database. This separation makes the code clearer and makes it easier to find and change certain pieces of functionality.

The dedicated service layer for recommendation logic is a very important part of the backend architecture. The project doesn't put all of the gap-detection and task-scoring behaviour directly into controllers. Instead, it puts this logic in a separate service module. This was a good choice for the architecture because the recommendation engine is the most unique and logically complicated part of the app. The code is easier to read, test, and add to when it is separate from basic request handling. It also shows good design because the recommendation rules are based on business logic, not presentation logic or direct access to the database.

GapMap uses SQLite as its database system at the persistence level. The schema is written in SQL and has the structures needed for tasks, timetable entries, and data relationships that help. SQLite was a good choice for this project because it lets you store data permanently on your computer without the hassle of setting up a bigger database server. For development, a local database file is enough to show how a full-stack application can handle real data, structured storage, and persistence. The database makes sure that tasks and timetable sessions entered by users stay available even after the application is refreshed or restarted. This is important for the application to work as a real productivity tool.

The system's architectural flow is clear and easy to understand. The frontend gets information from users and sends requests to the Express API. The backend then checks and processes those requests. It calls model functions to store or get data from the database and the service layer when it needs to make recommendations. When processing is done, the backend sends the results back to the frontend in JSON format. The frontend then displays those results to the user. This makes it easy to go from interaction to logic to persistence and then back to display.

An additional benefit about this architecture is that it makes it easier to keep up with. It is easier to improve one part of the system without affecting the rest because the responsibilities are clearly defined. For instance, you can make changes to the interface in the React frontend without changing the database schema. You can also make changes to the recommendation logic in the service layer without having to redesign the routing structure. This modularity is very helpful in iterative development, where features need to be changed often as testing shows new problems or chances to make things better.

Using a structured full-stack architecture also helps the project's academic goals. It shows that the system is more than just a mock-up of an interface or a prototype that isn't connected to anything else. Instead, it shows how to put together a working application by combining frontend rendering, API communication, business logic, and persistent storage. This makes the project technically significant while still keeping the implementation simple enough to handle.

4.3 Interface design

The GapMap interface was built on the ideas of clarity, hierarchy, and consistency. These rules were especially important because the app has a lot of different features that work together, such as task management, timetable management, dashboard summaries, and recommendation output. Putting all of these features on one busy screen would have made the system harder to understand and less useful. Because of this, the final design is based on pages, with each page having one main purpose. This method makes it easier to learn how to use the app, cuts down on visual clutter, and helps users get a better idea of how the system is set up.

One important design choice was to use a top navigation bar with clear and descriptive page names. This helps users find their way around the app by letting them go directly from one main area to another without getting lost. It was very important to give pages clear names because GapMap is meant to be used quickly and often. People shouldn't have to figure out what abstract labels mean or remember hidden navigation patterns. The navigation, on the other hand, shows the app in a clear and direct way, making it easier to tell the difference between the Home, Dashboard, Timetable, Tasks, and Recommendations pages.

Another important choice for the interface was to use a centred content column with cards that looked the same. Limiting the main content to a balanced central area makes it easier to read, especially on wider screens where layouts that aren't limited can look weak and hard to scan. Consistent card styling also makes the application's visual system stronger by putting related information into clearly defined sections. This makes the interface more organised and helps users tell the difference between different types of content without having to rely on too much decoration.

Forms were also made to be easy to read. Users could easily see what information was needed when adding tasks or timetable sessions because the inputs were grouped and clearly labelled. This was important because productivity tools need to let people interact quickly and easily. Users should be able to quickly enter information without having to deal with confusing field names or a form structure that doesn't make sense. Putting similar inputs together also makes the forms more logical, which makes them easier to fill out and check before sending.

The interface also has action buttons that look different depending on what they do. This helps people understand the difference between normal actions, important actions, and harmful actions. For instance, you can make primary actions like adding data stand out more clearly, and you can make deletion controls easier to see so that they are less likely to be triggered by mistake. This style helps usability by making it easier to understand what actions mean at a glance and by making sure that the meaning of different controls stays the same.

Another design choice was to use cards to show tasks, schedule sessions, and make recommendations. This kind of application works well with card layouts

because they let you separate pieces of information without losing the overall structure of the page. Cards make it easier to show metadata like title, duration, priority, and deadline in a way that is easy to read. They help make scheduled information clear for timetable entries. For suggestions, they give a structured space to show the suggested task, the gap that was found, and the reasons for the suggestion. This shared presentation style makes the app more consistent and cuts down on the amount of relearning that users have to do when they move between sections.

The dashboard summary cards are another important part of the final interface. These cards give you a general idea of how the system is doing right now, like what tasks are stored, what work is still to be done, what timetable entries are missing, and what gaps have been found. Users can quickly get a sense of the application's overall state by looking at these small summary blocks instead of having to look at each page in detail. This helps the system's goal of being more productive by making important information easy to find right away.

The typography and spacing of the app were kept simple on purpose. The interface doesn't use a lot of decorative elements to guide the eye; instead, it uses sectioning, colour contrast, repetition, and card structure. This was a good choice for a productivity app because the focus is on clear interpretation rather than fun or expressive visual design. A very decorative interface may draw attention, but if used too much, it can also get in the way of important tasks. So, GapMap uses a more controlled and academic visual style that helps with trust, readability, and focus.

During development, accessibility was considered in a practical way. It was easy to see which fields corresponded to which form labels, error messages were not hidden, action buttons were easy to tell apart, and content was easy to read on smaller screens. These decisions made the system easier to use for more people and less likely to cause confusion when people used it normally. These practical steps did make things easier to get to, but a more formal accessibility audit would still be helpful in the future. For instance, more testing of keyboard navigation, contrast ratios, semantic markup, and how well the screen reader works could make the system even stronger.

4.4 Data Design

The system keeps three main types of information: information about users, tasks, and timetable entries. It also looks forward to feedback on the recommendations later.

Entity	Key fields	Purpose in system	Validation / constraint notes
Task	title, description, estimated_duration, priority, energy_level, deadline, status	Stores the user's work items and all attributes required for recommendation ranking.	Title and duration required; deadline cannot be earlier than current date; enumerated values constrained to valid sets.
Timetable entry	title, day_of_week, start_time, end_time, location, entry_type	Stores the user's scheduled sessions so real gaps can be calculated.	Day and times required; start time must be earlier than end time; entry type restricted to valid values.
User / feedback support	user id plus optional recommendation feedback fields	Provides a default development user and supports later expansion into multi-user or recommendation-feedback features	Foreign keys maintain consistency between task, timetable and feedback records.

These fields were chosen because they are useful, not because they look nice. You need to know the estimated duration, priority, energy level, and deadline in order to score a recommendation. To find gaps, you need to fill out the day and time fields. Status fields are needed to filter out work that has been finished.

4.5 Design Rules and Constraints

The design of GapMap was based on a number of rules and restrictions. These rules weren't just there to protect the system from technical problems; they were also necessary to make it useful, reliable, and in line with its purpose. The quality of the outputs is directly affected by the quality of the inputs because GapMap uses stored data to make recommendations. If bad or illogical data gets into the system, the recommendations it makes are less reliable, and the app as a whole is less useful.

Because of this, clear rules for validating tasks, timetable entries, and recommendation generation were put in place.

The system puts a number of limits on tasks to make sure that the work items that are stored are useful and can be planned. A task title must not be empty; an unnamed task would not give the user any useful information and would not be able to be understood correctly in later views. The recommendation engine uses duration data to decide if a task is realistic for a given timetable gap, so you need to give an estimated duration as well. Also, the length must be within a reasonable range so that the system isn't messed up by values that are too high or too low. Deadlines also need to be checked. A deadline can't be earlier than the current date, because past deadlines don't help with planning for the future and would make the data less reliable. Finally, the database only stores valid values for priority, energy level, and status, so that only recognised and consistent categories are kept.

The main task-related rules can therefore be summarised as follows:

- title must not be blank
- estimated duration must be present
- duration must be within a sensible range
- deadline cannot be earlier than the current date
- invalid priority, energy level and status values are rejected

To keep the quality of the schedule, there needs to be a similar set of rules for timetable entries. The title of a timetable entry can't be empty because the user needs to be able to easily tell what the session is for. There must be a day, a start time, and an end time for a timetable entry to make sense. The system also makes sure that the start time is always before the end time, which stops sessions that don't make sense from being saved. Only valid weekday values are allowed so that the entries stay in line with what the recommendation logic expects. Only valid entry types are allowed so that the timetable data stays consistent and reliable.

The timetable-related rules are therefore:

- title must not be blank
- day, start time and end time must be provided
- start time must be earlier than end time
- only valid weekday values are accepted
- only valid entry types are accepted

The recommendation system also works within a set of rules and limits that are easy to understand. This is GapMap's most unique feature, so it was very important that its behaviour stayed believable and based on real-world planning. When making

recommendations, only real gaps between sessions are taken into account. We don't include fake gaps at the beginning and end of the day because they were found during development to make suggestions that weren't realistic and lower trust in the system. You shouldn't use the feature to look at tasks that have already been done. The point of the feature is to help users finish tasks that are still open. Tasks that take longer than the available time are also turned down because suggesting work that can't reasonably fit into the time available would make the app less useful. Finally, the recommendation engine can choose options that are not only possible but also make sense in context based on a combination of duration fit, priority, deadline, and energy level.

The recommendation-related rules are:

- only real gaps between sessions are considered
- fake start-of-day and end-of-day gaps are excluded
- completed tasks are not recommended
- tasks that exceed the available gap are rejected
- ranking depends on duration fit, priority, deadline and energy level

These limits were important because the quality of the data that the final system stores and processes have a big effect on how useful it is. If the input is bad, the output will be bad. If a task has no real duration, a timetable entry has wrong times, or a recommendation is based on a fake gap, the app's credibility will go down. The system keeps the database safe and the user experience good by making sure that these rules are followed.

The design rules also helped make sure that everything on the stack was the same. Validation on the front end makes the system easier to use by giving users faster feedback. Validation on the back end protects the system even if checks on the client side are skipped. This layered approach is very important for a full-stack project because it makes sure that the interface isn't the only thing that matters for correctness. Because of this, GapMap's rules and restrictions serve both a practical user-centred purpose and a technical reliability purpose.

5 SYSTEM DEVELOPMENT

5.1 Front-end Development

React and Vite were used to make the front end of GapMap. We chose this combination because it allows for component-based development, quick local iteration, and a clear separation between the structure of the interface and the logic behind it. This was a good choice for a project like GapMap, which has a lot of connected but separate features. It let the user interface be split up into pages and reusable parts. Because of this, important parts could stay separate and easy to understand, while common things like forms, cards, and navigation could still be used throughout the system.

At the entry point of the frontend, main.jsx is in charge of loading the main stylesheet and mounting the React app. This file makes the first link between the React component tree and the browser. App.jsx is also the main controller for rendering pages and moving around on them. This means that, in practice, it is the main point from which the whole frontend structure is coordinated. The system keeps the flow of page-level rendering clear and easy to manage by keeping this role at the top level.

We also made a special Navbar.jsx component to make sure that navigation is the same throughout the app. This component is very useful because it lets users easily switch between the main parts of the system and highlights the page that is currently active. Using a shared navigation component makes the interface more consistent and cuts down on the need to repeat navigation logic or styles in different places. It also works with the page-based design style used throughout GapMap, where each page has a clear main purpose.

The core pages of the frontend are:

- Home.jsx
- Dashboard.jsx
- Timetable.jsx
- Tasks.jsx
- Recommendations.jsx

This structure was helpful because each page is linked to a specific goal for the user. The Home page tells you what the app is about, the Dashboard gives you a general overview, the Timetable page keeps track of scheduled sessions, the Tasks page keeps track of work items, and the Recommendations page shows you how to use your free time. This way of organising the frontend made it easier to use and improved the development workflow. The app was easier for users to understand because each page had a different purpose. From the point of view of development,

it was easier to build and test one part of the application before moving on to the next. This fit with the project's iterative approach.

The form components were very important for the frontend implementation. TaskForm.jsx was made to get user input for making tasks and check it right away before sending it to the backend. TimetableForm.jsx does the same thing for timetable entries. These parts were needed not only to get information, but also to make the system easier to use. A clear form structure, grouped inputs, and local feedback help users know what they need to do and make it less frustrating when they make mistakes. Catching obvious mistakes early on the client side makes the whole interaction better and makes the app feel more responsive.

The pages for tasks and schedules also have list views that show saved entries after they are made. This was a big decision in terms of design and implementation because it makes new data visible right away. From the user's point of view, this makes them feel like they are getting a quick response and confirmation. Seeing a task or session right away in the interface helps build trust that the system is working as it should. This kind of immediate visible feedback is helpful in a productivity app because it lowers uncertainty and makes the workflow go more smoothly.

The recommendations interface was made to mix explanation with actionability. This part of the frontend is very important because it's easier to trust recommendation systems when they show not only what has been suggested but also why it has been suggested. In GapMap, GapCard.jsx shows the free time window, and RecommendationCard.jsx shows the suggested task, its main features, and the reason for the match. This division of responsibility makes things clearer and makes it easier to understand the recommendation output. Instead of showing a suggestion as a system decision without any explanation, the interface gives the user more information that helps them understand why it was made.

The dashboard was added as a summary layer that combines data from different parts of the system. It gets data from several sources and shows counts next to a preview of the best suggestion at the moment. This makes the app feel more complete because it lets the user see the whole system at a glance. The user doesn't have to go through every page to see what's stored; they can just look at the dashboard to get a quick overview of the timetable activity, task status, and recommendation output. This fits well with the app's focus on productivity because it makes important information easy to find without requiring extra work.

5.2 Back-end Development

Node.js, Express, and SQLite were used to make the backend of GapMap. Its function in the system is more than just storing and retrieving data. The backend is in charge of checking incoming data, coordinating business logic, enforcing application rules, and making recommendation output based on stored tasks and timetable

entries. It also handles requests and sends back responses. This makes it a key part of the whole system, since the app's quality, reliability, and usefulness depend a lot on how well the backend handles data.

Server.js is in charge of starting the server and setting up the database layer at the beginning of the backend. This file makes sure that the application is ready to handle requests and that there is persistent storage available before the rest of the system starts up. App.js also sets up middleware, registers the route structure, and handles basic errors. These two files set up the runtime environment for the rest of the application logic, which is what the backend is built on.

To keep the code organised and maintainable, the backend was structured into four main layers:

- **routes**, which define the available endpoints
- **controllers**, which validate input and coordinate behaviour
- **models**, which perform database access
- **services**, which contain non-trivial logic such as recommendation scoring

This layered structure was an important design choice because it made it clear what each part of the codebase was responsible for and lowered the risk of logic getting mixed up. The system is easier to understand, fix, and add to because each layer has a specific job. Instead of putting all of the request handling, validation, SQL operations, and recommendation logic in one file, each part of the backend is put where it makes the most sense.

The route layer sets the public structure of the API. It decides which endpoints are available and which controller functions should take care of them. This makes the endpoint design clear and makes it easier to see how the front end and back end talk to each other. The controller layer is behind the routes and is in charge of handling requests in a way that makes more sense. Controllers check the data that comes in, use application rules, call model or service functions as needed, and then send structured responses back to the client. In other words, controllers are the main link between raw HTTP requests and the application's deeper logic.

The model layer is in charge of getting to the database. Model files have the SQL code that lets SQLite read from and write to files. Putting this logic in models makes things more consistent and stops database queries from being spread out across controllers and other parts of the system. This makes it easier to keep the backend up to date because changes to how storage works can be made in a more focused way. For instance, the task routes in the task subsystem define the structure of the endpoint, taskController.js handles request validation and response handling, and taskModel.js contains the database queries. This pattern makes it clear who is responsible for what and makes it easier to understand each file.

The timetable subsystem works in the same way as the route-controller-model. This consistency is helpful because it makes the project easier to read and makes it easier to add new features in the future. After you understand how one subsystem works, you can read the parts of another subsystem that are the same more quickly. From a software engineering point of view, this makes the whole application feel more coherent and less frictional in development.

The service layer, and especially recommendationEngine.js, is the most important part of the backend from a technical point of view. This module has the more complicated business logic that makes GapMap different from a regular CRUD app. The recommendation engine doesn't just let the system create, read, and delete stored records. Instead, it processes timetable and task data to come up with useful suggestions for how to spend free time. Putting this logic in a separate service file was a smart design choice because it kept controllers from getting too complicated and made it easier to improve the recommendation feature on its own.

recommendationEngine.js has helper functions for a number of important tasks. These include changing time values into minutes so that time comparisons can be made consistently, formatting time ranges for output that users can see, sorting gaps by type or length, figuring out what time of day is likely to have the most energy, scoring tasks against a given gap, finding real free gaps between timetable sessions, and coming up with the best recommendations along with other options. Each of these functions plays a role in the overall recommendation process, which shows that this feature relies on more than just matching records. It needs you to think about how suitable, timely, and important something is.

This service layer took the project beyond just basic CRUD functions. When making the recommendation engine, I had to think carefully about how to show time, how to rank tasks that are competing with each other, and how to tell the difference between output that is only technically correct and output that is really helpful to the user. A task may be able to fit into a free period mathematically, but that doesn't mean it should be the most highly rated suggestion. So, the recommendation logic had to think about more than just whether a match was possible. It also had to think about whether it made sense in terms of how long it would take, how urgent it was, how important the task was, and the user's situation.

Validation is another important job of the backend that helps keep the system safe. Some checks are also done on the front end to make sure that the database is usable. However, backend validation is still important because it makes sure that invalid or inconsistent data can't get into the database even if client-side protections are turned off. This means looking for bad task values, impossible timetable ranges, and other inputs that would make the system less reliable. The backend helps keep a reliable data layer by enforcing these rules on the server side. This is especially important in an app where the quality of recommendations depends directly on the quality of the data stored.

5.3 Validation and Error Handling

Validation and error handling were very important in making GapMap because the system's value depends a lot on the quality of the data it gets. GapMap makes suggestions based on task and schedule information, which is different from a simple storage app. This means that wrong input can not only affect one record, but it can also make later outputs less accurate, realistic, and trustworthy. For instance, if a task is given the wrong amount of time, if a deadline is set in the past, or if a timetable session has start and end times that don't make sense, the recommendation engine may suggest things that seem technically correct but are actually wrong. Because of this, validation was seen as a core requirement instead of a small extra feature.

Validation was done at more than one level to keep the system safe. Forms give users instant feedback when they enter incorrect information at the frontend level. This makes it easier to use because mistakes can be fixed before the data is sent to the server. Immediate validation makes the application feel more responsive, cuts down on frustration, and stops people from trying to submit things that don't need to be. In GapMap, this means checking for missing titles, time ranges and deadlines that are impossible to meet, and deadlines that are earlier than the current date. By catching these problems early, the interface helps the user enter correct data and lowers the chance that obvious errors will get to the backend.

But frontend validation isn't enough on its own in a strong system. People can get around client-side checks on purpose or by accident, so the application needs to protect itself on the server side as well. Because of this, the same rules are checked again in the back end. This is a key part of full-stack design because it makes sure that the database is safe even if the browser interface isn't. Backend validation is the last line of defence for the integrity of a system. It stops invalid or inconsistent data from being stored. In GapMap, this method was especially important because the reliability of the recommendation output depends directly on the reliability of the stored task and timetable data.

The rule about deadlines is one of the best examples of this layered validation strategy. The design of the date input and the form logic on the front end of the task form stops the user from choosing a date in the past. This makes the interface more helpful and helps the user enter the right information. In taskController.js, however, this check happens again, where submitted deadlines are checked again and thrown out if they are earlier than the current date. This double protection is useful because it protects both the data that is stored and the experience of the user. It makes sure that the rule is always followed, no matter how the request gets to the server.

The same idea works for other types of input as well. Timetable entries must have valid titles, valid values for the day of the week, and reasonable start and end times.

Tasks must have a realistic length, as well as known priority, status, and energy values. The system lowers the chance of corrupted or misleading data getting into the recommendation process by carefully applying these checks. This helps both technical reliability and user trust because users are more likely to trust recommendations when they think the data behind them has been handled correctly.

During development, error handling got better bit by bit. In earlier stages, some problems only showed up when requests failed or refresh behaviour broke. This showed how important it was to make mistakes more clear. As the system got better, it became clearer and more helpful how to deal with problems. Rather than allowing the application to fail silently, the backend and frontend were adjusted so that meaningful messages could be returned and displayed. Responses like "invalid task id," "invalid deadline," or "failed fetch" are some examples. These messages are helpful for both developers and users. They made it easier to find and fix bugs during development. In final use, they help the user understand what has gone wrong instead of being left with unclear or confusing behaviour.

Trust in the system is also helped by good error handling. It should be easy to understand and predict how a productivity tool will work. When something goes wrong, users should get feedback that is clear and useful, not vague or hard to find. This rule was very important for GapMap because the app has a lot of different layers that are all connected, such as forms, API communication, backend logic, and database access. Messages that are easy to understand help keep trust between these layers and make the system as a whole feel stronger.

5.4 Key Features Implemented

5.4.1 Task creation and management

One of the most important parts of GapMap is the task creation and management feature. This is because it gives the system the structured work data it needs to make recommendations later. This feature is not just for making a list of things to do; it is also for storing work items in a way that is detailed enough for the system to figure out if they fit into certain gaps in the schedule. In this way, the task subsystem is very important to the overall goal of the application because the quality of the recommendations depends directly on the quality and structure of the task data that the user enters.

From a technical point of view, TaskForm.jsx on the front end is where task creation starts. It takes the user's input and does immediate validation checks before sending it. After the form data is ready, it goes through the central API layer in api.js, which sends the request to the backend. On the server side, taskController.js checks the request again, applies the rules that need to be followed, and makes sure that the creation process runs smoothly. If the data is valid, the request goes to taskModel.js, which puts the task in the SQLite database and keeps it there for a long time. This

layered flow makes sure that task creation works and is easy to fix and keep up with because each part of the system has a clear job to do.

One big problem in this area was that a GapMap task needs more metadata than a regular to-do list item. A task manager usually only needs a title and maybe a due date for a task. But for GapMap to give useful suggestions, it needs more information. So, each task needed fields like estimated duration, energy level, priority, and deadline. This is because these values have a direct effect on how well a task fits into a free gap in the schedule. For example, the system needs to know how long the task will take to see if it can fit into an open window, and priority and deadline help it decide how urgent the task is. Energy level also affects the quality of recommendations by making it easier to match tasks to the right times of day.

This extra difficulty made it even more important to check the tasks. The recommendation engine won't work as well if these values are missing, wrong, or not realistic. Because of this, the process of making tasks was set up to not only store information but also keep it safe. There must be titles, the times must make sense, the deadlines must not be in the past, and only accepted values can be used for fields like priority, status, and energy level. These checks make sure that the tasks that are stored are still useful both as individual tasks and as inputs to the recommendation logic.

With the finished feature, users can add tasks in a meaningful and organised way, see them clearly in the interface, delete them when they are no longer needed, and change their status from pending to completed. This helps the app work better as a whole and gives the user an organised and up-to-date view of their workload. So, the final product is more than just basic CRUD functions; it's a task management system made just for planning based on recommendations. This is one of the most important features that were added to GapMap in its final version.

5.4.2 Timetable management

Managing schedules is another important part of GapMap because it gives the schedule context needed to find gaps and make suggestions. The system uses task data to figure out what the user needs to do and timetable data to figure out when the user is busy and when they are free. This feature is necessary for the app to find useful free time and link tasks to real job opportunities. Because of this, managing schedules is not an extra feature of the system; it is an important part of the overall design.

Timetable sessions are added to the front end through TimetableForm.jsx. This form lets the user fill in the important information needed to set up a scheduled session, such as the title, day, start time, end time, and type of entry. The data is sent through the API layer to the backend, where it is checked in the controller layer before being sent to the right model to be stored in SQLite. After being stored, timetable entries are retrieved and shown in an ordered list so that the user can easily see their current schedule in the app. This implementation has the same layered structure as

the rest of the project, with tasks split up between the frontend, API communication, controller logic, and persistent storage.

One of the biggest problems with this feature was that the recommendation engine's usefulness depends directly on how accurate the timetable is. If the timetable data is missing, wrong, or not valid, the outputs that come later will not be reliable. For instance, incorrect start and end times could make false gaps, and malformed weekday values could stop sessions from being grouped or processed correctly. Because of this, it was very important to check the timetable. The system had to make sure that all the required fields were there, that the start times were before the end times, and that only valid weekday values and entry types were allowed. These checks were necessary not only to keep the data safe, but also to make sure that the recommendations made from that data were still good.

Another important part of managing the timetable was how sessions were shown after they were made. Showing entries in an ordered list makes it easier for the user to understand the stored schedule and lets them know right away that the system has recorded the session correctly. This also helps people trust the app because they can see that their timetable data is being handled correctly and will still be there when they need it. This kind of visibility is useful in a productivity app because it makes the link between what users do and how the system works stronger.

The end result is that the system can safely store timetable sessions and use them to find gaps. This feature gives the recommendation engine the structural time-based information it needs to work properly. Because of this, it is a key part of making GapMap more than just a task manager; it can now actively look at schedules and help people use their time better when it is broken up.

5.4.3 Recommendation engine

The recommendation engine is what makes GapMap stand out the most. It's also what makes it different from a regular task manager or calendar app. The goal is to turn raw data about schedules and tasks into helpful suggestions that take into account the user's situation and help them decide how to use their free time. This feature doesn't just show a list of tasks that need to be done or a list of scheduled sessions; it connects the two and gives useful information based on how the user plans their day. By doing this, it solves the main problem that was identified at the beginning of the project: helping users make good use of short, broken-up periods of time.

From an implementation perspective, the recommendation engine operates through several stages of processing. First, timetable entries are grouped by day so that the system can evaluate availability within a meaningful schedule structure. Then, these entries are cut down to the chosen active day window, which makes sure that the engine only looks at the part of the day that the user thinks is important for work. After this, the engine only finds real gaps between sessions instead of assuming that all free time is automatically useful. The system looks for real gaps and then

compares them to the stored task list. It then rejects any tasks that don't realistically fit into the time that is available. Then, based on a number of factors, such as how long they will take, how important they are, how much energy they will take, and how urgent the deadline is, the remaining tasks are scored. This creates a list of possible recommendations in order of importance, from which the best one and other options can be chosen.

This part of the application was harder to make than the other main parts because it needed the system to do more than just basic create, read, update, and delete operations. It had to think about time, compare the features of different tasks, follow rules consistently, and make output that the user would understand. The recommendation process, in particular, needed to find a balance between different types of suitability. A task might be a good fit in terms of how long it will take, but it might not be the best choice if its deadline is far away or if it requires more energy than the time of day when you are most likely to do it. This meant that the recommendation engine was the most important place for business logic and user-centred judgement to work together.

During development, a big problem came up when the first version of the engine, even though it worked technically, made gaps that weren't realistic. It made big windows at the beginning and end of the day, which made the feature less believable from the user's point of view. This made it clear that there is a big difference between output that is technically possible and output that is actually useful. The system could make suggestions, but the suggestions weren't very helpful because the time windows didn't show real work opportunities. Because of this, the feature didn't have the level of credibility that a productivity system needed at first.

This problem caused the recommendation logic to be greatly improved. We changed the gap-detection process so that only real gaps between real sessions were taken into account. We also used the active day window in a more controlled way to cut timetable data without making fake edge gaps. These changes made the engine much more realistic. Recommendations became more believable, more useful to the user in real life, and more in line with the app's intended use. This process of making small changes over time was especially important because the recommendation engine is the most important part of GapMap.

The end result was that the later version of the recommendation engine became the app's best and most unique feature. It does a good job of turning stored schedule and task information into useful suggestions that take into account the time available, the nature of each task, and the larger goal of helping people manage their time better. This took the project far beyond what a normal productivity tool would do and gave GapMap a clear and unique functional focus.

5.4.4 Dashboard summary

The dashboard summary feature was made so that users could quickly and easily see the current state of the system. The goal is to show the user all of the most

important information from the application in one place so they can see their stored workload and the context of the current recommendation without having to go to each page. This kind of summary layer is useful in a productivity app because it helps the system make sense of things quickly and gives it a stronger sense of coherence. The app starts to feel more like a single tool with a clear overall picture instead of a bunch of separate pages.

From a technical point of view, the dashboard works by making several API calls at the same time and putting the data that comes back into a simple interface. It collects summary data like the number of tasks, the amount of work that needs to be done, the total time spent on each task, and the information about gaps that it finds. Then it shows these numbers on separate cards. The dashboard has a preview of the strongest current recommendation in addition to these statistical summaries. This lets the user not only see how much data is stored in the system, but also figure out what the application thinks is the best way to use the time it has right now. The end result is a page that works as a high-level summary instead of a detailed management view.

One of the biggest problems with adding this feature was that it needed a lot of other parts of the system to work right first. The dashboard could only give useful information if the task storage, timetable management, and recommendation generation were all working well. This meant that the dashboard couldn't be built on its own; it had to be part of something else. It didn't do anything else; instead, it brought together the results of earlier development work. This dependency meant that the quality of the dashboard was closely linked to how stable and correct the whole system was. If there is a problem with the API responses, the stored data, or the recommendation logic, the dashboard output will not be useful right away.

Because of this dependence, the dashboard was also a good way to test full-stack integration. It helped make sure that the different parts of the application were always working together by pulling data from many different endpoints and showing it to the user all at once. In this way, the dashboard was useful not only for users, but also as a sign of how complete the whole system was. If the dashboard could show accurate summaries and a believable recommendation preview, this meant that the data flow throughout the application was working as it should.

In the end, the dashboard makes GapMap seem more complete, informative, and connected. The app would still have useful individual features without it, but it would feel more like a collection of pages and less like a whole. The dashboard makes it easier to use by putting important information all in one place. It also gives the user a better idea of their current situation at a glance. This makes the app seem more like a single productivity system instead of a bunch of separate feature pages.

5.5 Version Control and Workflow

During the development of GapMap, GitHub was the main way to keep track of changes. Version control is important for more than just working together in software development. It is also important for keeping track of progress, protecting work, and keeping a clear record of how a project changes over time. GitHub gave GapMap a useful way to keep track of both the technical development and the overall progress of the system.

One of the best things about GitHub was that it kept a record of the project's history that was easy to see. Every commit made a note of what had been changed and when those changes had been made. This made it easier to see how the system changed over time and to tell the difference between the main stages of the build. For instance, the first commits could show how the schema and backend were set up, while the later commits could show how recommendation logic, interface wiring, validation improvements, and layout changes were added. This visible history was helpful not only during development but also for giving a clearer picture of how the project had moved forward.

GitHub also helped keep track of important milestones. As the app grew from a simple CRUD app to a more complete full-stack system, commits served as natural checkpoints that showed important steps in the work. This made it easier to tell when big new features were added, when the structure was improved, and when later tweaks were made. In this way, version control helped the project not only with technical issues, but also as a way to manage the project in a light way. It made the development story more coherent, which is useful for both personal organization and school submissions.

Another big benefit was the ability to make safe restore points. In iterative development, especially on a project that includes frontend, backend, and recommendation logic, it's common for new changes to cause problems that weren't there before. GitHub made it safer to try new things because if something went wrong, you could still use older, stable versions of the code. This was especially useful when GapMap went from just storing tasks and schedules to making more complicated changes, like updating backend validation and fixing the recommendation engine. In these later stages, being able to go back to a working state from before made development safer and more controlled.

This also made it less likely that mistakes would cause damage. Without version control, it can be hard to find and undo a change that didn't work, especially if you've changed a lot of files. On the other hand, GitHub made it easier to compare versions, find out when an error was made, and fix problems without hurting the rest of the codebase. This made the development process more confident because changes could be made knowing that the system's previous working state had been kept.

Version control also helped make the submission story for both the software and the project documentation around it clearer. A well-maintained repository shows that development is ongoing, not just a single final upload. This makes the project more credible as a real process of design, implementation, and refinement. This is especially helpful for an academic computing project because it shows how the system was built step by step instead of just showing up as a finished product with no visible development history.

The repository also helped with better project management and keeping things clean. The choice to stop keeping track of the live SQLite database file was a helpful one. Taking this out of version control cut down on unnecessary noise in the repository and made the source tree cleaner and more focused on the code and documentation. This is a good development practice because files that are created or specific to an environment are often better left out of tracking if they don't help you understand the source code itself.

Repository used during the project:

https://github.com/Tayef006/GapMap

6 TESTING

6.1 Testing Strategy

Testing was an important part of the GapMap project because the app works as a connected full-stack system instead of a set of separate interface pages. The system's value depends on more than just whether each feature seems to work on its own. It also depends on whether the whole path from user input to backend processing to stored data to final recommendation output works correctly and consistently. If there is a problem at any point in this chain, the whole application may not work as well. Because of this, testing was seen as a necessary part of quality control during development, not as an optional last step.

The testing strategy combined several different forms of checking so that both technical correctness and practical usability could be examined. These included:

- manual functional testing
- integration testing across frontend and backend
- validation testing for incorrect input
- persistence testing using SQLite
- UI and usability inspection
- scenario-based testing of recommendation behaviour

The main part of the testing process was manual functional testing. This meant checking to see if the system did what the functional requirements said it should do, like making tasks, showing stored data, deleting records, changing the status of tasks, adding timetable sessions, and making recommendations. The application was tested in the same way that a normal user would use it by directly testing each of these behaviours through the interface. This was important because it made sure that the main question was whether the system really worked as it should, not just in theory.

Also, integration testing between the frontend and backend was needed because a lot of the problems that came up during development didn't show up until different parts of the application were linked together. For instance, a form might look right on the front end, but it could still fail if the API request was wrong or if the controller handled the data in an unsafe way. If the frontend didn't show the returned results correctly, then successful backend logic wouldn't be very useful. Integration testing, then, looked at how the whole system worked together to make sure that requests, responses, rendering, and stored data all worked as they should.

Another important part of the plan was validation testing for wrong input. GapMap needs accurate task and schedule data to make good suggestions, so it was important to see how the system dealt with bad values. This meant making sure that empty titles were not accepted, impossible timetable times were not allowed, invalid category values were not accepted, and deadlines that were earlier than the current date could not be saved. The project used a layered validation method, so these tests were done at both the front end and back end. This was important because a strong application shouldn't depend on the client side alone to keep the data layer safe.

It was also necessary to test persistence with SQLite because the app's usefulness depends on data staying available after a refresh or restart. Testing in this area made sure that tasks and timetable entries were still stored correctly in the database and could be accessed in later sessions. If GapMap didn't have persistence, it would only work as a temporary interface and not as a useful productivity tool. So, checking that the database storage worked was an important part of making sure the project worked as a real full-stack application.

Along with checking for feature correctness, the testing plan included checking the UI and usability. This meant making sure that pages were easy to read, that navigation was clear, that action labels were clear, that forms were easy to fill out, and that visual layout choices helped people understand rather than confused them. This kind of check was important because an application can meet its technical requirements but still be hard to use. GapMap is meant to help people interact with each other quickly and easily every day, so testing the interface was an important part of judging the quality of the final system.

Scenario-based testing of recommendation behaviour was a very important area. The recommendation engine is the most unique part of GapMap, so it needed to be tested for both technical performance and how useful and realistic it is. We used different combinations of timetable sessions, task durations, deadlines, priorities, and energy levels to see if the recommendations made sense in context. This kind of testing was very useful because it found problems that wouldn't have shown up with just simple CRUD testing. For instance, earlier problems like gaps that lasted all day became clear only when recommendation behaviour was looked at in real-life situations.

The current minimum viable product does not include automated testing, which is still a problem for the project. Automated tests would have made the system even stronger by letting important logic, validation rules, and API behaviour be checked over and over again. They would have also made it easier to make changes later on because regressions could have been found faster. But even though there was no automated testing, the manual testing that was done during the project was well-organised and closely related to the functional requirements. So it wasn't just random clicking through the interface. Instead, it created a meaningful and organised way to

check for quality that helped find problems, make sure features were complete, and support ongoing improvement.

6.2 Functional Testing

We did functional testing on GapMap while it was being developed to make sure that the main parts of the system worked as they should. The application was built to meet a specific set of functional requirements, so testing in this area was mainly about making sure that each user-facing feature worked as expected and that the results were what was expected. This was especially important because GapMap is a full-stack system that is connected. A feature isn't complete until the whole chain of interaction, processing, and display works right.

The main functional areas tested during development were:

- adding a task
- viewing tasks
- deleting a task
- changing task status
- adding a timetable entry
- viewing timetable entries
- deleting a timetable entry
- generating recommendations
- applying a day window to recommendations
- refreshing pages and confirming persistence
- checking dashboard counts

Test area	Scenario / action	Expected result	Outcome
Add task	Submit a valid task with title, duration, priority, energy level and future deadline.	Task is stored, success feedback is shown and the task appears in the list.	Passed after task refresh issue was solved.
Delete task	Delete an existing task from the tasks page.	Task is removed from the database and no longer displayed in the list.	Passed.
Update task status	Toggle a task between pending and completed.	Status changes are stored and completed tasks are excluded from recommendations.	Passed
Add timetable entry	Submit a valid timetable session with day and time range.	Entry is stored and displayed in the timetable list	Passed.
Delete timetable entry	Delete an existing timetable session.	Entry is removed and list refreshes without stale data.	Passed.
Generate recommendations	Provide at least two sessions on one day and one or more pending tasks.	The system displays a real gap and recommends a task that fits it.	Passed after real-gap logic replaced edge-of-day gaps.
Persist active day window	Change the recommendation window and refresh the page.	The previously selected day window remains visible after reloading.	Passed after localStorage was added.

Test area	Scenario / action	Expected result	Outcome
Dashboard summary	Open the dashboard after adding tasks	Counts reflect stored data and the best current suggestion card is populated when suitable data exists.	Passed.

Once the main controller and recommendation problems were fixed, these tests showed that the core application flow was working well.

6.3 Validation Testing

Validation testing was a very important part of the GapMap project because the recommendation engine only works well if the data stored in the system is good. In GapMap, bad input can have bigger effects than in a simple application, where bad input might only affect one record. If the deadlines are wrong, the durations are missing, or the timetable times are wrong, the recommendations that come later may not be as realistic or trustworthy. This is why validation testing was used to make sure that the system always rejected bad data and kept both usability and data integrity safe.

The main validation cases tested during development were:

- deadline earlier than today
- deadline equal to today
- missing title
- missing estimated duration
- invalid duration values
- timetable start time later than end time
- blank timetable fields
- invalid weekday or entry type

Validation case	Input / action	Expected result	Outcome
Past deadline	Submit a task with a deadline earlier than the current date.	Submission is blocked and an error message is shown.	Passed on both front-end and back-end.
Today's deadline	Submit a task with the deadline equal to the current date.	Task is accepted because the deadline is still valid.	Passed.

Validation case	Input / action	Expected result	Outcome
Missing task title	Submit the task form without a title.	Submission is blocked with a message explaining the issue.	Passed.
Invalid duration	Submit a non-sensical duration such as 0 or a value above the supported limit.	Submission is blocked.	Passed.

Validation case	Input / action	Expected result	Outcome
Timetable end before starting	Submit a session where the end time is earlier than the starting time.	Submission is rejected.	Passed.
Missing timetable fields	Leave required day or time fields empty	Task is accepted because the deadline is still valid.	Passed

One very important rule for this project is that deadlines cannot be set before the current date. Both the client and server now follow this rule, which makes them both more usable and more stable.

6.4 UI and Usability Testing

UI and usability testing looked at whether the final system was easy to understand, use, and navigate in real life. This kind of testing was important because GapMap is meant to help people make quick decisions and do everyday tasks. The application would still be less effective if users found the layout confusing, the forms awkward, or the recommendation output hard to understand, even if the underlying features worked correctly. Because of this, usability testing was used to find out not only if the system worked, but also if it made its purpose clear and helped users feel confident.

The main areas considered during UI and usability testing were:

- clarity of page names
- readability of lists and cards
- ease of using forms
- visibility of success and error feedback
- visual balance of the layout
- whether the recommendation explanation felt understandable and trustworthy

A key element to look at was how clear the names of the pages were. Because the app is made up of pages instead of one dashboard for all of its features, it was important that users could quickly figure out what each section was for. Testing looked at whether page titles like Home, Dashboard, Timetable, Tasks, and Recommendations clearly showed what they were meant to do. This was important because good page names make things less confusing and help users understand how the system works.

We also looked at how easy it was to read lists and cards. GapMap shows a lot of its information in card-based layouts. This includes tasks, timetable sessions, summary data, and suggestions. So, usability testing looked at whether these cards were easy to scan, whether the information hierarchy was clear, and whether users could quickly find the most important information. This was important because the app is meant to be easy to understand at a glance, not slowly studied.

Another important area was how easy it was to fill out forms. Because entering tasks and schedules is such an important part of the system, the forms had to be easy to use and not too complicated. Testing looked at whether the labels were clear, the fields were logically grouped, and the whole submission process felt easy. This was

especially important because using the form over and over would make the app less useful as a daily productivity tool.

We also looked at how easy it was to see feedback on success and failure. People need to know if something worked, didn't work, or needs to be fixed. Because of this, testing looked at whether feedback messages were clear, easy to read, and sent at the right time. Clear feedback makes people surer of the system and less likely to get confused after doing important things like filling out a form or fixing bad input.

Another vital component of usability testing was checking the visual balance of the layout. How the content is arranged on the page affects how it looks, how easy it is to read, and how good it seems. Tests showed that the first layout felt too wide and too left-aligned, especially on bigger screens. This made the interface less balanced and made some pages look less polished than they should have. In response, the layout rules were changed so that the main content area could only be a certain width in the centre. This made it easier to read and made the app look more professional.

A very important usability issue was whether the explanation of the recommendation made sense and could be trusted. The recommendation engine is what makes GapMap stand out, so it was important that users could understand why a certain gap was suggested for a certain task. Testing looked at whether the explanation gave enough background information to make the recommendation seem real instead of random. This became especially important when earlier versions of the system made gaps that weren't realistic. This made people less trusting, even when the code was working properly.

The results of UI and usability testing were mostly good, especially after a few changes were made during development. The new layout width made the navigation clearer, the new recommendation explanation made the system easier to understand, and the new layout width made the system look better. But this testing also found a number of important problems. At first, the main content looked like it was left-aligned and too wide. The recommendation gaps didn't seem real until the logic was changed, and the active day window had to be entered again and again until client-side persistence was added. These problems didn't always mean that the technology failed in the strictest sense, but they did make the user experience worse.

This was a noteworthy outcome of the testing process. It showed that the most useful results weren't just about crashes or broken features. Problems with clarity, realism, convenience, and user trust were just as important because they made the app feel less usable. Trust and understandability are very important for a system like GapMap to work well, since its main goal is to help people make better decisions.

6.5 Test Results and Fixes

Testing during the development of GapMap not only confirmed that features were functioning, but also uncovered significant flaws that compromised reliability, realism, and usability. These problems were important because they showed that the app's quality didn't just depend on adding features; it also depended on improving them based on how people actually used them. Each problem that was found was fixed in a specific way, and then the system was tested again to make sure that the change had made it better. This process was an important part of the project's iterative development method.

One of the first significant issues discovered concerned the task creation workflow. In this case, the task creation worked, but the task list didn't refresh right away. This made things confusing for the user because the interface seemed to confirm success but then failed to show the new data. An investigation revealed that the controller tried to access req.body.user_id during a GET request, even though req.body might not be defined in that situation. This meant that the problem wasn't with adding tasks; it was with how the next retrieval request was being handled. To fix this, optional chaining was added when getting the user ID so that the controller could handle GET requests more safely. After retesting, both creating a task and fetching it worked correctly, which showed that the workflow was now reliable.

A second important issue affected the recommendation engine. Early testing showed that the system was generating unrealistic giant gaps, which weakened the credibility of the recommendation feature. The problem was that the gap-detection logic saw the rest of the day as one valid recommendation window instead of only looking at real free periods between real timetable sessions. The engine was technically making output, but the suggestions that came from it didn't seem real to the user. This was a big problem because the recommendation engine is the app's most unique feature, and its value depends a lot on how realistic it is. To fix this, the gap-detection logic was changed so that it only finds the gaps between sessions that come one after the other. When we tested it again, we found that the recommendations were much more realistic and better suited to the system's intended purpose.

Another fault identified through testing involved deadline validation. At one stage of development, the system allowed the user to enter deadlines that were earlier than the current date. This was a problem because deadlines that have already passed aren't helpful for planning and make people less likely to trust the task data that has been stored. The problem happened because there was no rule at the frontend or backend level to stop these kinds of values from being used. To fix this, both sides of the system had to follow strict deadlines. We made changes to the frontend so that it wasn't easy to choose past dates from the form. We also made changes to the backend so that it would reject invalid deadlines if they were sent directly. Retesting showed that deadlines that had already passed were always rejected. This fix was

very important because it made the stored data better and made the recommendation output more reliable in the future.

Another drawback that came up during usability testing was the general layer of the interface. Even though the app worked, the main content area looked unbalanced because it got too wide and looked like it was left-aligned on bigger screens. This made the system look worse and made it harder to read some pages comfortably. The issue was not with the functionality being broken, but with how it was presented and how clear it was. To fix it, the layout rules were changed so that the main content area had a set maximum width and a centred container. After retesting, the interface looked more balanced, more professional, and easier to understand. This improvement showed that the results of the tests weren't just about technical problems; layout and readability were also important to the app's overall success.

These fixes show that the testing process had a direct effect on how well GapMap works. Each problem that was found showed a flaw that hurt either trust, realism, correctness, or usability. Each fix made the final system stronger in a useful way. The retesting stage was very important because it showed that the changes had fixed the original problem instead of just changing behaviour without any clear benefit.

6.6 Summary of Testing

The testing phase showed that GapMap's core minimum viable product works well in all of the main areas where it is meant to be used. The final system was able to support the important workflows that the project needs to work. Users can add and remove tasks, add and remove timetable entries, change the status of tasks, stop invalid deadlines from being stored, get recommendations based on real timetable gaps, and see summary information through the dashboard. These results show that the MVP's main functional requirements were met and that the system works as a connected full-stack application instead of just a bunch of separate features.

One of the greatest things about the testing process was that it had a clear and direct effect on decisions about development. Testing was not seen as a way to confirm something after it had already been put into use. Instead, it was an active part of the development cycle and helped make both the logic and the way the system looked better in a way that made sense. Testing found problems like tasks not refreshing correctly, gaps in recommendations that weren't realistic, weak deadline validation, and an unbalanced layout. These problems were then fixed through targeted refinement. This shows that testing helped find bugs and also made the final application better and more trustworthy.

Another positive aspect about the testing phase was that it looked at more than just finishing features. We checked not only functional behaviour, but also validation rules, persistence, usability, and how realistic the recommendations were. This broader approach was especially important for GapMap because the app's success

depends not only on whether the actions work technically, but also on whether the outputs are believable, clear, and useful. In this way, testing helped make the system more stable and more reliable.

The biggest problem with the testing phase was that there were no automated tests. Even though the manual testing was well-planned and closely related to the functional requirements, automated testing would have made the project even better by letting people check important logic, validation rules, and API behaviour over and over again. It would have also made it less likely that changes made later would cause problems. A second problem was that formal user evaluation was only done on a small scale. The project did not include a large-scale structured user study with multiple external participants, even though usability was carefully checked during development. This means that the evaluation of user experience was mostly based on testing and improvement done by developers instead of feedback from a wider range of people.

Even with these problems, the testing process was thorough enough to stabilise the MVP and show major flaws in both the system logic and the interface design. It showed clearly that the main features work together correctly and that the final application is a good working prototype. In general, the testing phase was a big part of GapMap's success because it made the system more reliable, easier to use, and better suited for its purpose.

7 ANALYSIS

7.1 How well GapMap solves the problem

GapMap effectively solves the original problem at the level of a minimum viable product. The main problem that came up at the start of the project was that a lot of students know how much work they have to do, but they have trouble using short, broken-up periods of free time between scheduled sessions. Most tools only let you store tasks or plan your calendar, but they don't connect the two in a way that helps you figure out what you can realistically do in a certain amount of time. GapMap solves this problem directly by putting task management, schedule data, and recommendation logic all in one system.

One of the main ways the app helps is by making it easier to organise tasks. Instead of having work spread out across memory, notebooks, messages, or other digital tools that aren't related, GapMap lets users keep track of their tasks in a structured and consistent way. This makes it easier for the user to see what needs to be done and lowers the chance that work will be forgotten or not given the right amount of importance. The system makes planning and making decisions more reliable by putting all tasks in one place.

The app also makes it easier to see deadlines. Deadlines are linked to tasks, which makes it easier to see how urgent they are and helps the user remember what work is coming up. The system also ignores deadlines that are no longer useful, like dates that are earlier than today. This makes the stored information better and stops the system from being hurt by bad planning data. Because of this, GapMap doesn't just keep track of tasks; it keeps them in a way that makes them more useful for managing your time.

But the most important way that GapMap solves the original problem is through its recommendation feature. The system helps the user make better use of broken-up time by looking at real gaps between timetable sessions and matching them with tasks that fit. This is what makes GapMap more than just a simple to-do list. A regular task manager may remind the user that there is work to do, but it usually lets the user choose when to do that work. GapMap goes a step further by using the timetable context to suggest what could actually be done in the time that is available. This is how the application helps people take action instead of just being aware of something.

This makes the system especially useful for students who have busy schedules, deadlines that overlap, and short breaks between commitments. In these cases, the problem is often not knowing that work is out there, but figuring out if a short amount of free time is worth using and what task would make sense during that time. This is where GapMap can help. It makes planning easier by suggesting tasks that fit the

gap, and it also makes it more likely that time that would have been wasted can be used productively.

Nevertheless, the system doesn't work all the time. GapMap still needs good input from the user. The system's recommendations are less useful if a task is given an unrealistic amount of time, if its deadline is wrong, or if there are missing or incomplete entries in the timetable. This is a big problem because the app doesn't make planning better on its own; it just makes it easier for the user to act on good data. To put it another way, GapMap helps with planning, but the user still has to enter the right information.

Even with this flaw, the MVP shows that the project does a good job of solving the original problem. It doesn't try to automate every part of time management, but it does offer a useful and focused improvement over other methods that don't work together. GapMap is a better way to deal with fragmented free time than just a regular task list or timetable tool. It does this by centralising tasks, making deadlines more clear, and making suggestions based on real gaps in your schedule.

7.2 Analysis of design decisions

Several design choices made during the creation of GapMap turned out to be very successful, both from a technical point of view and in terms of the system's final quality. These choices affected not only how quickly the project could be finished, but also how reliable, believable, and easy to maintain the final product was. In many cases, the best results didn't come from adding more features; they came from making good structural choices that helped the project's main goal.

The decision to use JavaScript across the whole stack was the first one that worked out well. Using JavaScript for both the front end and the back end made development easier and more organised. This made it easier to switch between client-side and server-side work because the same language, similar syntax, and shared programming mindset could be used throughout the system. This was especially appropriate for a project of this size because it gave us more time to work on the application logic and make it better instead of having to switch between different languages and development styles all the time. It also helped the codebase be more consistent and made it easier to understand the whole system.

Another good choice was to use a backend-first implementation order. The project didn't start with interface polish; instead, it focused on the database schema, the backend structure, and the core application logic. This was a big decision because GapMap relies on how tasks, timetable entries, and recommendation behaviour work together. If the system had been built with looks in mind first, it would have been easier to make an app that looked good but didn't work well. The project made sure that the later interface was built on top of a real working system by setting up the

schema and server-side logic before focusing on presentation. This helped stop shallow development and helped make a more complete full-stack product.

The third very important choice was to change the recommendation logic so that only real gaps between timetable sessions were taken into account. This was one of the most important conceptual changes made during the project. The previous method could technically make recommendations, but it thought that big, fake parts of the day were real gaps, which made the suggestions less believable and useful. Changing the logic so that only real free periods between sessions were used was a big step away from output that was only technically possible and toward output that was actually useful. This change made the recommendation feature much more in line with what the app was originally meant to do, and it also made GapMap stand out more as a realistic productivity tool.

A fourth successful choice was to use validation in more than one place, especially on both the front end and the back end. This made the project better in two important ways. First, it made the system more robust by making it hard for invalid data to get in, even if client-side checks were skipped. Second, it made users more likely to trust it by giving them instant feedback in forms and making sure that the backend always followed the same rules. This layered validation strategy was especially useful in a system like GapMap, where later recommendations depend directly on the quality of stored input. It helped keep the database safe and the interface easy to use.

These design choices worked because they helped the project at a structural level instead of just at a surface level. Using one language across the stack made development faster, working backend-first made the technical substance stronger, refining recommendation logic made the practical realism better, and layered validation made the system more trustworthy and reliable. All of these choices helped make the final system work and made sure that GapMap became a more useful, coherent, and maintainable app.

7.3 Technical challenges and difficulties

From the point of view of a second-year computer science project, GapMap is a good and believable level of technical challenge. The system is more than just a basic coursework prototype because it can do more than just one interface and a simple set of create, read, update, and delete operations. Instead, it puts together a lot of different technical issues into one full-stack application that is connected. This makes the project more complex and gives it more depth than a typical CRUD-based task manager.

The project is technically better than a basic app because it has both a full front end and a full back end. The front end shows information, lets users interact with it, and makes requests. The back end checks input, coordinates logic, and sends back structured responses. This separation means that the project needs to work on more

than one level instead of just one script or interface. This means that the developer has to keep track of both how features work and how different parts of the system talk to each other.

The app also has persistent relational-style data storage, which makes things even more complicated. SQLite stores tasks and timetable entries so that they are always available, even after you close the app. This means thinking about how to design the schema, how to keep the data safe, and how people will access the database. Using persistent storage makes the project more realistic and useful from a technical point of view because it acts like a real application instead of just a temporary demo.

Asynchronous communication between clients and servers is another problem. The frontend and backend do not work together as a single process. Instead, they send and receive data through API requests. This means that the system needs to be able to handle asynchronous operations correctly, manage how things load and refresh, and make sure that the interface stays in sync with the state of the backend. It can be hard to find faults in this area because they can come from request timing, controller logic, data format, or rendering behaviour instead of one specific place.

The project is also more advanced than a regular task manager because it has rule-based recommendation logic. GapMap doesn't just store and show information; it also processes that information to make it useful. This recommendation logic made the system look at tasks and available time slots, throw out options that didn't work, and rank the remaining candidates based on a number of factors. This adds a much stronger element of reasoning than a system that is only based on storage.

The problem of time-based reasoning is related to this. The app has to figure out how to read timetable sessions as structured periods, find the real gaps between them, and figure out if a task can fit into those gaps. It can be harder than it seems to handle time in software because calculations have to be consistent, edge cases have to be handled carefully, and outputs still have to make sense from the user's point of view. This became especially important in GapMap when the first version of the recommendation logic made gaps that weren't real. To solve this problem, the project had to go beyond just coding and start thinking more carefully about what made a job offer believable.

Layered validation is another important technical feature. There is checking of input in both the frontend and the backend, not just one place. This makes the system more stable and is a better way to do software engineering, but it also makes development more difficult because the rules have to stay the same across all parts of the stack. So, the project needed not only to write validation logic, but also to design it so that it didn't hurt usability or the integrity of the database.

Adding dashboard aggregation gives the project even more technical depth. The dashboard isn't a static page; it needs other parts of the system to work right and pulls together information from different sources to make a high-level summary view.

This needs APIs to work together, data to flow correctly, and features that used to be separate to work together reliably. As a result, it serves as both a feature for users and a test of how well the overall application architecture works.

Also, the project has a user interface that can handle multiple pages and is aware of the current state. The system is organised into several dedicated pages, each with a specific function. However, the navigation, data updates, and displayed results all work together to create a smooth user experience. Active navigation highlighting, persistent day-window preferences, and immediate list refresh behaviour are all features that make the frontend more interactive than a single-page static interface.

The recommendation engine is the most technically interesting part of the project. Even though it is based on rules instead of machine learning, it still needed careful thought about how well it fit, how well it ranked, when it should be used, and what users expected. The engine had to figure out not only if a task could be suggested, but also if it should be suggested instead of other options. This meant making sure that the identified timetable gaps were realistic while also balancing the length of time, priority, deadline urgency, and energy level. In this way, the recommendation engine gave the project a unique and technical identity that a simple task manager wouldn't have.

Another hard part of the project is keeping the data flow consistent across the whole stack. The application needed architectural thinking, discipline for validation, interface design, and debugging across many layers. A change in one part of the system could have an effect on another part without directly affecting it. This meant that developers often had to follow behaviour across the frontend, API, backend controllers, service logic, and database. This kind of reasoning across layers is a big part of what makes the project technically sound at the second-year level.

8 EVALUATION

8.1 Evaluation of the final product

The last version of GapMap can be seen as a strong and trustworthy minimum viable product. It works because it has all the features needed to solve the original problem, and it does so through a full-stack implementation instead of just a prototype or interface mock-up. The system is now functionally coherent, technically sound, and clearly in line with the goals set at the beginning of the project. Even though it isn't as broad as a finished commercial application yet, it shows a meaningful and well-executed way to solve a real-world productivity problem.

The clear focus on problems is one of the best things about the final product. GapMap was made to fix a specific problem with other productivity tools from the start: it was hard to make good use of short periods of free time between scheduled sessions. The project's focus stayed the same, which kept the app from getting off track or adding too many features that weren't related. The end result, then, has a purpose. The main goal of each major feature is to help users plan their work and make better choices about how to use their time.

Another big plus is that the system can be used on all levels. GapMap isn't just a bunch of screens that don't connect to each other or a static interface. It has a structured API layer, recommendation logic, and persistent storage in SQLite. The frontend is built with React, and the backend is built with Express and Node.js. This makes the project more technically credible and shows that the person understands how different parts of a web application work together. The system is more than just a simple interface-driven coursework project because it works on multiple levels.

A further advantage about the final product is that it can store data for a long time. Instead of being stored in memory temporarily, tasks and timetable entries are kept in a local database. This makes the app work like a real productivity system. This persistence allows for continued use and makes the application more useful because information is still available after a refresh or restart. It also makes the project stronger from a technical point of view by showing that the system has a real data layer instead of just a front-end that looks good.

The product also gets a lot out of meaningful validation. This is a very important strength because GapMap needs good input data to make useful suggestions. The system doesn't just take any value that the user enters. Instead, it stops or rejects deadlines that are not valid, task information that is not complete, and timetable values that don't make sense. This makes the project more reliable and trustworthy for users, and it shows that the developers cared about data integrity as well as how it looks.

The recommendation feature is probably what makes the final product stand out the most. This is what makes GapMap stand out from other task lists and calendar apps. The system doesn't just keep track of tasks and sessions; it also looks at them all together and turns them into useful suggestions for how to use time that is broken up. This makes the app even more useful and unique. So, the best part of the product isn't the storage of data itself, but how it turns stored data into useful output. This feature gives GapMap the feel of a more unique and thoughtful productivity tool.

The interface is also positive because it is easy to read and looks good. The final design is simple, clear, and fits the type of app being made. Users can easily move from one page to another, know where to put information, and understand the dashboard and recommendation outputs without too much explanation. The interface isn't as polished as a commercial platform, but it's coherent and professional enough to support the system well and show that the design choices are good.

Even though it has some good points, the final product also has some flaws that show it is still an academic MVP and not a finished consumer app. One clear flaw is that there is no way to verify who you are. The current version of the system doesn't have user accounts or secure personal access, so it can only be used by one person at a time. A real product that is used in the real world would need authentication to protect privacy, identity, and user-specific data management.

The product also doesn't support collaboration or multiple users. This means that it can't be used right now for planning together, managing tasks together, or coordinating schedules between different users. This wasn't what the project was meant to do, but it would still be a problem if the system were to be turned into a more useful tool in the real world.

Notifications or reminder systems and live calendar synchronisation are two other features that are missing. These are common in many modern productivity tools and would be very useful in future versions. The MVP's main goals are not affected by their absence, but the product does rely on the user choosing to open and use the system instead of getting proactive prompts or automatic schedule updates.

Another problem is that there wasn't enough formal user testing done during the project. The system was tested in a systematic way and the usability was carefully looked at, but the evaluation did not include a wide-ranging structured study with many end users. This means that the product's usability has mostly been tested by developers and not by a lot of independent users. Consequently, certain conclusions regarding user experience remain tentative.

Lastly, the app's final interface doesn't yet have a fully polished edit workflow. There is logic for updating the backend, but editing is not fully and consistently available for all user-facing entities. This means that the final system is still missing some of the features that a mature productivity app should have, even though it has strong creation, deletion, and status management.

8.2 Evaluation of the development process

The way GapMap was developed can be seen as effective overall because it was iterative, focused, and closely tied to the real problems that came up during implementation. Instead of following a strict order in which all design decisions were made ahead of time, the project went through a cycle of building, testing, finding flaws, and improving the system in response. This method worked well for the application because GapMap relies on the interaction between several connected layers, such as frontend behaviour, backend logic, validation, and recommendation output. This meant that a lot of important problems didn't become clear until all the features were working together from start to finish.

One of the best parts of the process was deciding to start with the schema and backend foundations. Beginning with the data model and server-side structure helped make sure that the system was built on a solid and realistic foundation. This kept the project from getting too focused on looks before the underlying behaviour was set. This foundation-first approach was especially good because the recommendation feature needs to have accurate relationships between tasks, timetable entries, and stored values. It made sure that later work on the interface was based on a real working system and not just on how the pages looked.

The choice to build one usable vertical slice at a time was another strong point of the process. The features were added in a logical order that took dependencies into account, so that each new part of the system could build on something that was already working. For instance, the timetable support was put in place before the recommendation logic, and the task handling was working before the dashboard summaries were added. This step-by-step method lowered risk because there was usually a testable baseline at each stage of development. It also made debugging easier because problems were often easier to find because they were linked to the most recent feature instead of being buried in a lot of unfinished changes.

Frequent manual testing was also an important strength of the development process. Testing didn't wait until the end of the project; it was done all the time to make sure the application was working as it should. This made it possible to find real problems early on, such as problems with data flow, layout, validation, and the realism of recommendations. The back-and-forth between implementation and testing was especially helpful because some of the project's biggest improvements came from problems that were only found during real-world use. In this way, testing helped with development directly instead of being a separate last step.

Using GitHub throughout the development process was another good part of the workflow. People used version control as an active development tool instead of just a requirement for submission. This helped keep track of the project's history, gave people a way to go back to earlier versions when new changes caused problems, and made it easier to tell the difference between major steps in progress. This was especially useful for a project that went through many rounds of improvements

because it let them try new things without losing stable versions of the code from earlier. It also helped make the whole development process more organised and professional.

Another strength was the willingness to improve the most important part instead of letting it stay weak. The recommendation engine is what makes GapMap unique. The project got better because this feature was looked at again and improved when problems were found. The logic was changed so that recommendations were more believable and better fit with the project's original goals. This was done instead of accepting behaviour that was technically correct but not realistic. This shows that the development process was not only about finishing features, but also about how well those features worked in real life.

However some parts of the development process didn't go as well as they could have. One flaw was that some choices about how to design the interface were based more on how easy it would be to implement than on formal prototyping. The project did involve lower- and medium-fidelity thinking, but not every decision about the interface was completely worked out through structured design work before work on the project began. This meant that some problems with usability and layout didn't show up until later, after several features had already been linked. A more formal early prototyping process might have found some of these problems earlier and made it less necessary to change the interface later.

The lack of automated testing was another flaw. The manual testing done during the project was well-organised and helpful, but automated tests would have made the process much stronger. They would have made it easier to check important logic over and over, find regressions after changes, and feel more sure about things like validation and recommendation behaviour. The fact that they aren't there doesn't mean the project wasn't tested well; it just shows that there is a gap between the current MVP process and a more advanced development workflow.

The project also had few formal stakeholders involved. The system was made with students in mind, and usability was carefully thought out. However, the development process did not include a lot of structured feedback from a larger group of end users or outside stakeholders. Consequently, numerous design decisions were assessed predominantly via developer-driven evaluation and testing. At the MVP level, this was fine, but a more advanced process would benefit from more user feedback to directly test ideas about usability, usefulness, and real-world workflow.

Another problem was that some usability problems didn't show up until multiple pages were already linked together. This is what happens in full-stack development, but it also suggests that doing a workflow simulation or planning for usability more formally might have helped. Some issues, like layout balance, trust in recommendations, and having to set up the app over and over again, weren't noticed until the app was more complete. Even though these problems were eventually fixed,

they show that it's harder to judge the quality of an interface when the focus is first on technical implementation.

8.3 Limitations

GapMap works well as a minimum viable product, but it still has some major flaws that separate it from an academic prototype and a production-ready app. These limitations do not detract from the system's value or its effectiveness in resolving the initial issue at the MVP level. But they do show that the current version is still a student project that is focused on one thing, not a full commercial product that is meant to be used by a lot of people.

One of the biggest problems is that the system now assumes a default development user instead of allowing for real authentication. This means that the app still doesn't have safe user accounts, login features, or personalised access to protected data. Because of this, the current build is mostly good for testing and development, not for a real multi-user environment. In a production system, authentication would be necessary to uphold privacy, security, and individual data ownership.

In line with this, GapMap is currently a system that can only be used by one person. The design and implementation assume that one person is working with one set of saved tasks and calendar entries. There is no support for multiple users, separate accounts, shared spaces, or features that let people work together. This was fine for the project because the main goal was to show the main idea of productivity, but it would still be a big problem if the app were to be developed further.

Another problem is that there are no notifications or reminders. Right now, GapMap can store tasks, show deadlines, and make suggestions, but it doesn't actively remind the user or send them reminders about work that is coming up. This makes it less useful as a planning tool for everyday use, since many real productivity apps use notifications to remind users of deadlines and tasks that need to be done. So, the current version relies on the user to open the system and check their information by hand.

An additional issue is that entering tasks and schedules is all done by hand. Users can't import their sessions and tasks from other programs; they have to type them directly into the app. This is fine for showing how the system works, but it makes things less convenient and raises the risk of having incomplete or out-of-date data. The recommendation engine needs accurate input, so relying on manual data entry makes it more likely that the quality of the recommendations will go down if users don't keep their task and timetable records up to date.

There is also no third-party calendar integration, which is related to this. Right now, the system doesn't work with Google Calendar, Outlook, or university timetable platforms. This makes it harder for GapMap to fit into a student's normal workflow.

Calendar integration would be helpful for a more advanced version of the project because it would let schedule data be updated automatically and used more reliably to make recommendations.

The project also has a flaw in the evaluation area, where it has been more technically sound than empirically sound. The system was thoroughly tested for functionality, validation, persistence, and usability, but its evaluation has mostly been based on testing and analysis done by developers rather than feedback from a larger group of real users. This means that we know a lot about how the system works technically, but we don't have as much proof that it works in the real world as we would if we were studying a more mature product.

Finally, deployment for live, long-term use on multiple devices has not yet been fully discussed. The current version uses SQLite and is good for development and testing on your own computer, but it hasn't yet been turned into a fully deployed platform with strong hosting, remote persistence, and access from multiple devices. Even though the core system is functionally complete at the MVP level, this limits the app's practical readiness for wider use in the real world.

These limitations do not mean that GapMap was not a successful student project. They set the system's current limit instead. The app already has a clear idea, a working full-stack implementation, and a unique recommendation feature. However, it is not yet complete, integrated, or tested enough to be ready for production. In that way, these limits make it clearer what the project has accomplished and what needs to be done next for GapMap to move past the MVP stage.

8.4 Future Improvements

GapMap is a good start as a minimum viable product, but there are a number of realistic changes that could make it more useful for everyday use and expand its academic scope. These future changes would not change the main purpose of the system, but they would make it more useful, easier to use, and valuable in the long run. The current version already shows the main idea well, so the next step in development will be to make the app more complete, easier to use, and stronger as a real productivity tool.

Adding real user accounts and authentication would be one of the most important improvements for the future. Right now, the system assumes a default development user and doesn't let users log in securely with their own accounts. If you added authentication, users would be able to log in safely, keep their own tasks and schedule data private, and use the app in a more realistic multi-user setting. This would be a big step toward making GapMap a real product instead of just a project prototype.

Making the edit flows for tasks and timetable entries more polished would also be a useful improvement. The current system supports strong creation, display, and deletion behaviour, but editing is not yet fully exposed in a polished way throughout the final interface. Adding smooth and complete update workflows would make the app more useful for everyday use because users often need to change deadlines, durations, priorities, and session details after they first enter them. This would make the system more complete overall.

Adding notifications and the ability to set reminders would also make GapMap much more useful. Right now, the app only works if the user opens it to see tasks and suggestions. A reminder system could help users remember deadlines, work that needs to be done, and good ways to use their free time without having to check all the time. This would make the app more like what people expect from a modern productivity tool.

Importing or syncing calendars would be another important step forward. Right now, you have to enter timetable sessions by hand, which takes more time and makes it more likely that the schedule data will be wrong or missing. The app would be easier to use if it could connect to other calendar or timetable services. This would also make the recommendations better by making sure that the stored schedule is more accurate about real commitments. This would be very helpful for students who already use digital calendars as part of their daily work.

Other helpful changes could be the ability to set tasks to repeat, which would make the system better for academic or personal tasks that need to be done more than once, and drag-and-drop prioritisation, which would give users a more flexible and easy way to organise their work. These changes would improve the app's ability to manage tasks and make it feel more responsive to how different users act.

The recommendation feature itself could also be extended. One way to make things better would be to add more detailed recommendation feedback loops. This would let the system learn from whether users accept, ignore, or complete suggested tasks. This would make recommendations feel more personal over time, even if you don't need to use a full machine-learning approach. Analytics that show users how much time they saved or how many tasks they finished during schedule gaps could also help them feel like they are making progress and having an effect. This would show how well fragmented time is being used, which would make the system more valuable.

From an interface point of view, better mobile optimisation would be another good change to make in the future. Even though the current layout is still easy to read on most widths, the app was not fully developed as a mobile-first platform. Making the design better for smaller screens would make the system more useful for everyday use, especially for students who want to quickly check recommendations or enter tasks from a phone or tablet.

From a software engineering point of view, the project would also benefit from automated tests for the controllers and the recommendation logic. These would make the application more reliable, lower the chance of regressions happening during future development, and make it easier to keep up with as more features are added. The recommendation engine would especially benefit from automated checks that can be done over and over again because it has the most complicated and unique logic in the system.

Deployment with a persistent hosted database and public frontend/backend hosting would be a big step forward in the future. This would make GapMap more than just a local development environment; it would also make it a real online application. When combined with authentication and a better persistence layer, deployment would bring the system much closer to being a useful product in the real world instead of just an academic demonstration.

The combination of authentication, polished edit support, and reminders would probably be the most important of all these possible improvements. These changes would have the biggest effect on making GapMap something that users could count on all the time. Authentication would make the system private and safe, editing would make it more useful in real life, and reminders would make it more active than passive. These changes would make the app more useful in everyday life and show the clearest change from a student MVP to a more mature productivity platform.

9 CONCLUSION

This study examines fragmentation-based problems in time management through the creation and development of GapMap – a full-stack web application encompassing task management, timetable storage, and recommendation generation logic. The project is based on the problem statement that although people understand the volume of work, they often have trouble with the effective use of time periods between their activities. The current solutions allow people to save information about the tasks and/or to create a schedule, but they fail to provide assistance in deciding on how many tasks one may manage to accomplish within the limited period of time. Thus, GapMap was designed as a response to this problem.

In this case, the successful completion of the project proved the viability of solving the problem at hand using available means. The created application allows saving tasks, sessions of a user's timetable, organizing deadlines, and receiving recommendations concerning tasks one should accomplish during breaks in his/her daily schedule. The recommendation generation capability gives the product more meaning in contrast to a simple CRUD tool.

As far as technical skills are concerned, the project showed the author's proficiency in different substantive aspects of software development, including frontend, backend programming, database designing and validation, integration with API services, and recommendation generating rules. The project also required taking into account various aspects related to time, ranking, and application behaviour. As a result, GapMap is not only a functional application, but also a technically substantial second-year computing project.

All the stated goals were met as far as the main objective of the project was concerned. Specifically, it led to the development of a viable minimum viable product, the fulfillment of the core requirements, validation of its functionality, and implementation of the recommendation generating mechanism as the key distinctive feature of the product. In addition, the iterative nature of the process allowed improving the system substantially by addressing discovered defects and usability issues.

There are clear directions for further improvement associated with this project. Primarily, it relates to the necessity of advancing beyond the MVP stage and making the system more comprehensive. For example, the inclusion of authentication procedures, improved edit processes, reminder capabilities, and integration with the calendar would make the tool more useful for everyday life.

10 USER GUIDE

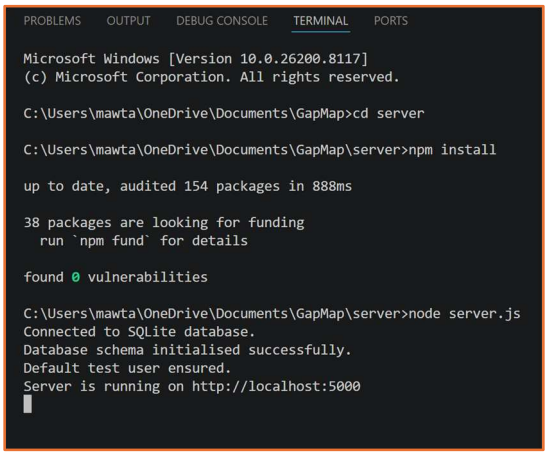
10.1 Accessing the system

10.1.1 Running the back end of the system

GapMap runs as a front-end and back-end system

To run the back end:

1. Open a terminal in the server folder using Command Prompt
   a) `cd server`
2. Install the required packages using npm
   b) `npm install`
3. Run the server using the line of code below:
   c) `node server.js`

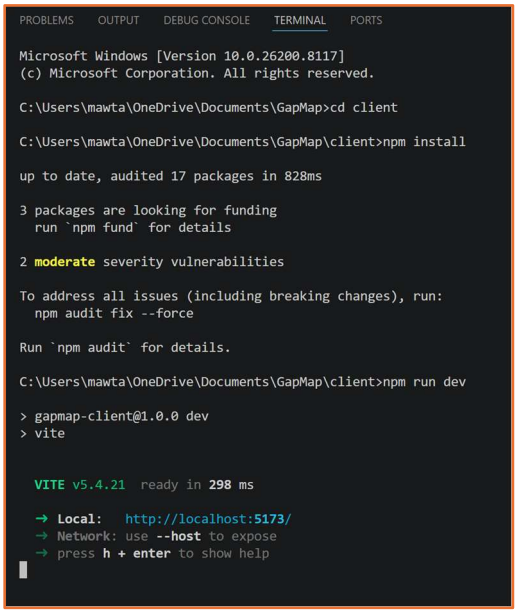


4. Wait until the server starts on http://localhost:5000 and Ctrl+click to run the back end.

When complete, a tab named “localhost:5000” will open on the default browser and will say:
`{“message”: “GapMap API is running”}`

10.1.2 Running the front end of the system

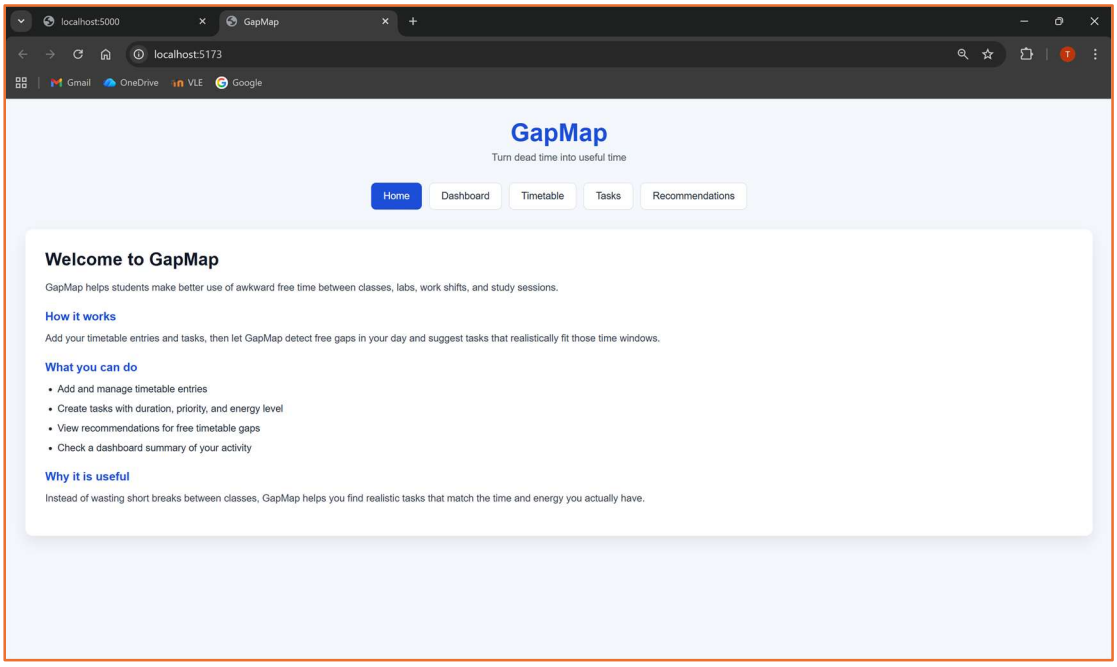
5. Open a second terminal in the client folder using Command Prompt.
   d) `cd client`
6. Install the required packages using npm
   e) `npm install`
7. Run the front end using the line of code below:
   f) `npm run dev`



8. Open the local Vite address, typically http://localhost:5173, with Ctrl+click

The Home page will now be displayed.

Note: The back end must be running at the same time as the front end, otherwise API requests will fail.

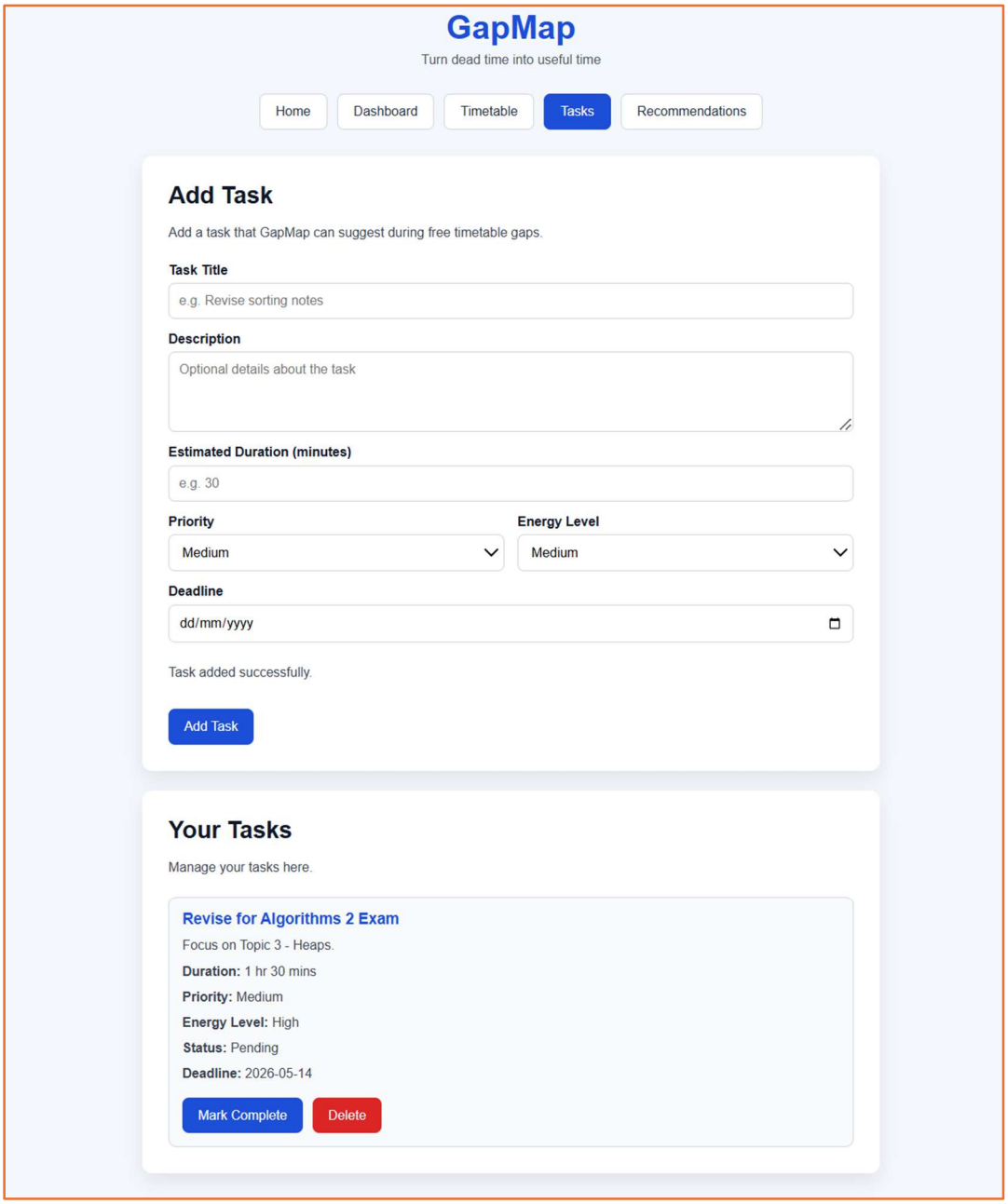


10.2 Creating a Task

To create a task:

1. Open the **Tasks** page using the navigation bar on the top.
2. Enter a task title.
3. Optionally enter a description.
4. Enter an estimated duration for the task, in minutes.
5. Choose a priority level.
6. Choose an energy level.
7. Select a deadline, if necessary.
8. Click **Add Task.**

If the input is valid, the task will be saved and displayed in the task list.

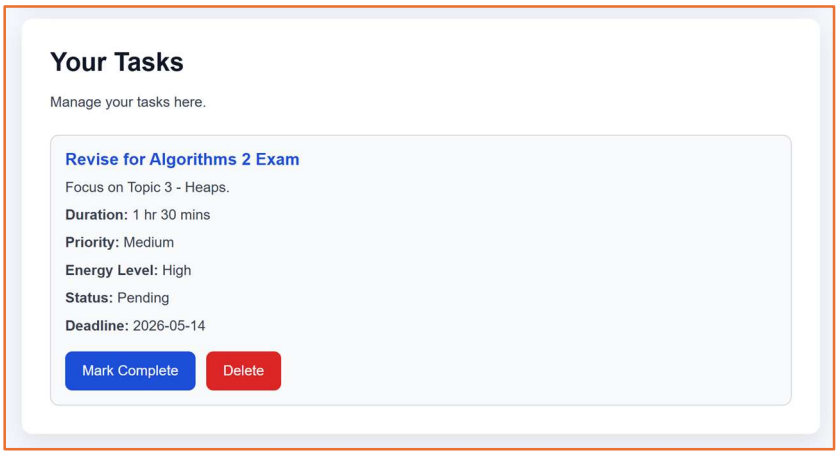


10.3 Editing and deleting tasks

The current interface allows the user to delete tasks and change their status from the task list itself. Full task editing capabilities have been incorporated into the back-end development and are thus a logical next step in terms of user interface development; nevertheless, they have not been showcased as an efficient editing page as yet.

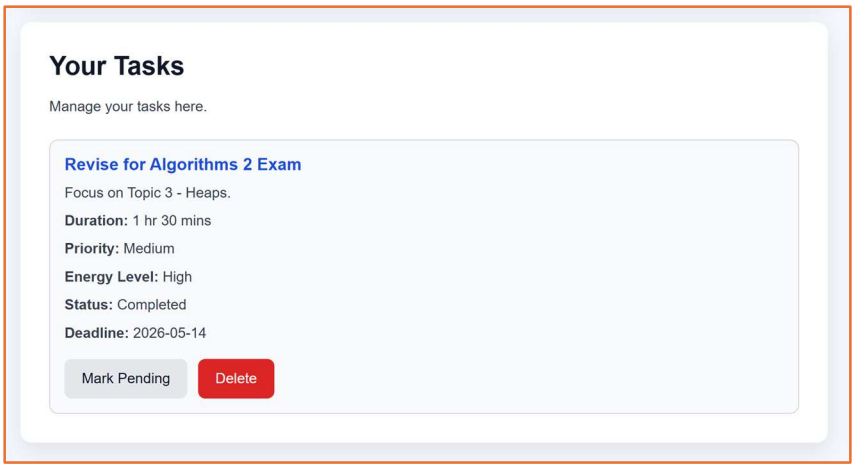
To delete a task:

1. Open the **Tasks** page using the navigation bar.
2. Locate the task card.
3. Click the red **Delete** button.



To change the task status:

1. Open the **Tasks** page using the navigation bar.
2. Locate the task card and click the blue **Mark Complete** button if a pending task is completed, or the grey **Mark Pending** button if the task is not yet completed.

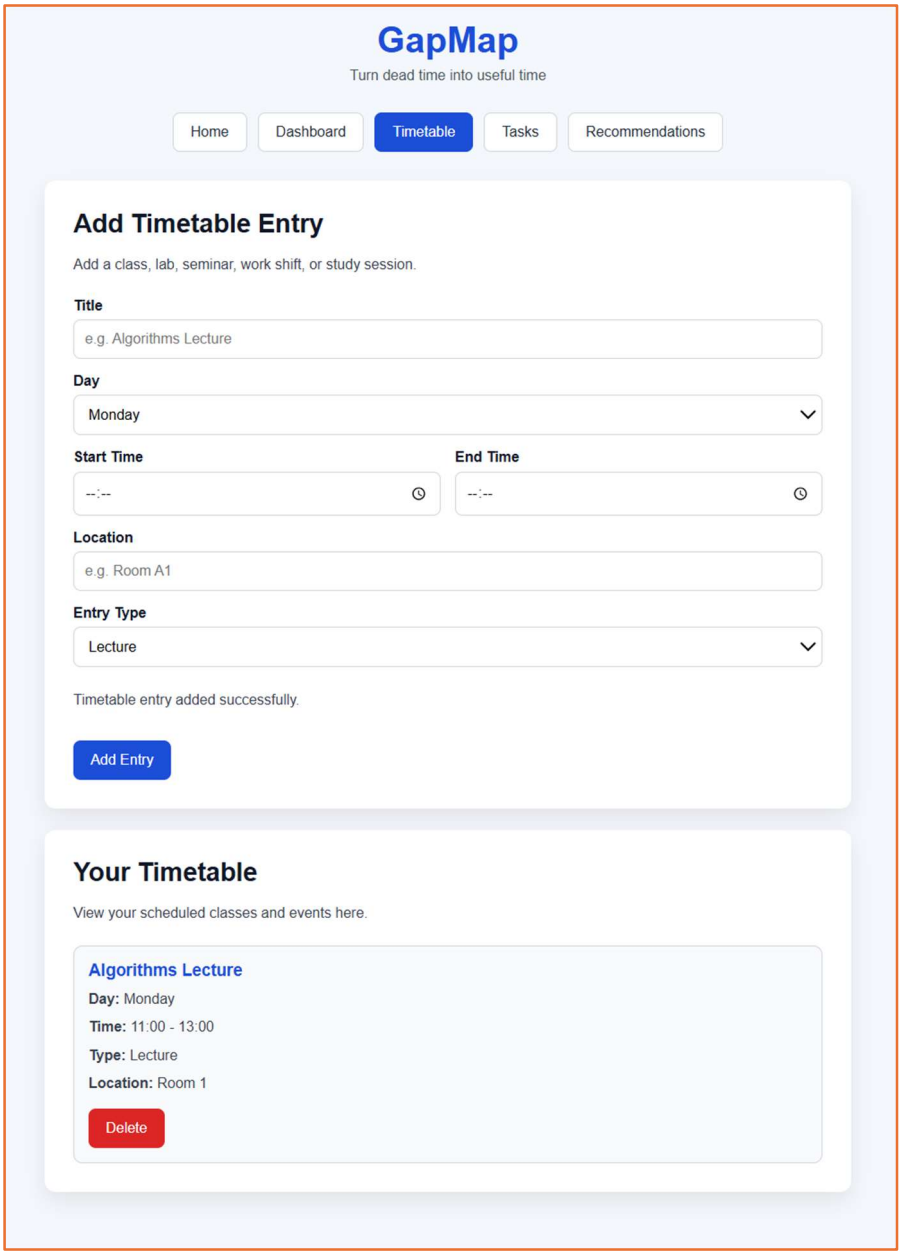


10.4 Creating a timetable entry

To add a timetable entry:

1. Open the **Timetable** page using the navigation bar
2. Enter a session title.
3. Select the day of the session.
4. Enter the start time in 24h format, *e.g. 1pm would be 13:00*.
5. Enter the end time in 24h format.
6. Enter the location of the session.
7. Choose the entry type, *e.g. Lecture, Seminar, Laboratory etc.*
8. Click **Add Entry**.

If the input is valid, the session will be saved and displayed in the “Your Timetable” list below.



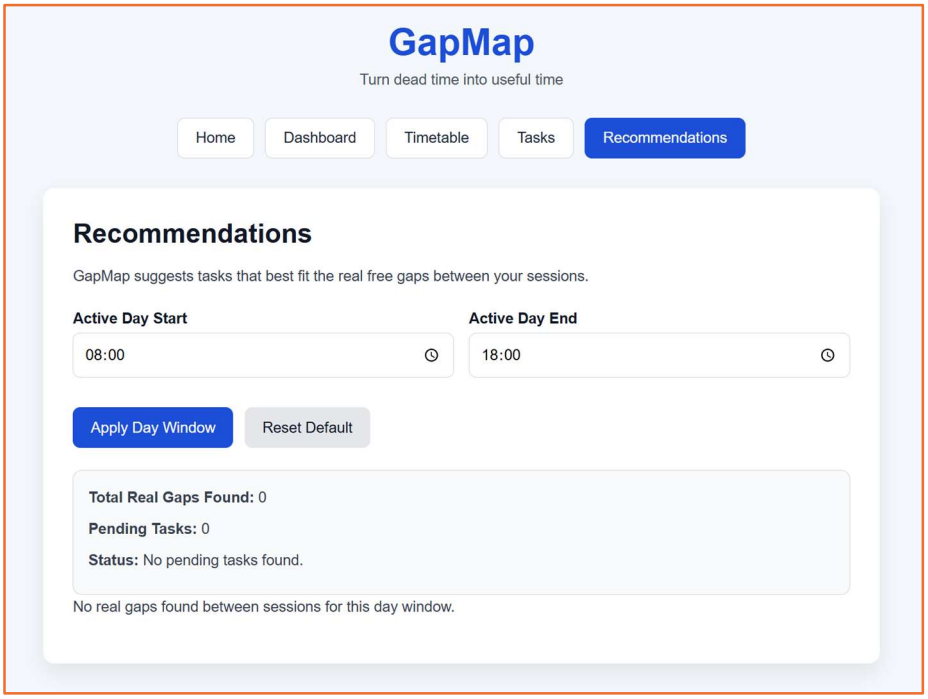
10.5 Adjusting the active day window

The active day window is representative of what times the user will be active in their studies per day.

To change the active day window:

1. Open the Recommendations page using the navigation bar.
2. Enter the starting time of your active day, in 24h format, *e.g. 1pm will be 13:00.*
3. Enter the end time of your active day.
4. Click “Apply Day Window”.
5. To reset the times to default, click “Reset Default”.

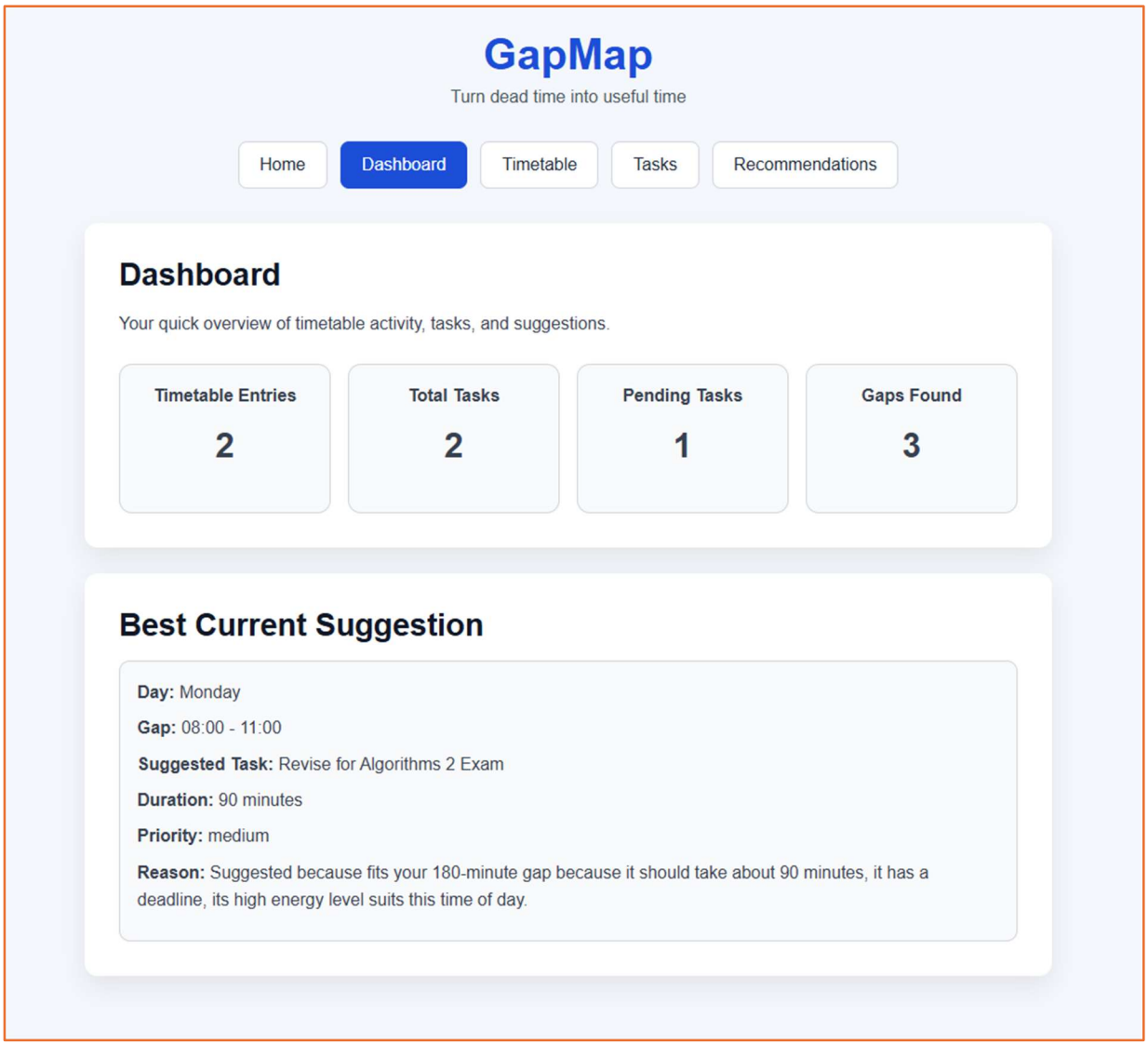
<u>Note: The default active day window is 08:00 to 18:00.</u>



10.6 Dashboard

The dashboard indicates the number of timetable entries, total number of tasks, number of pending tasks and gaps found between sessions.

It will also display a card with the best suggestion for a select period of time between two sessions.



10.7 Recommendations

As mentioned previously, the recommendations page allows the user to select an active day window for their studies.

Moreover, it identifies all the gaps available in a students timetable and fits in appropriate tasks which can be completed in the time frame. A match score is given which indicates how suitable it is for the user to complete the task at that given period of time. A higher match score means more suitability while a lower match score suggests the task is less suitable for this gap.

Image 1:

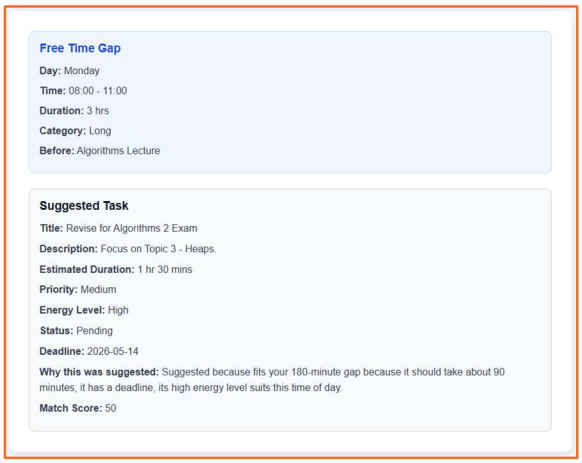


Image 2:

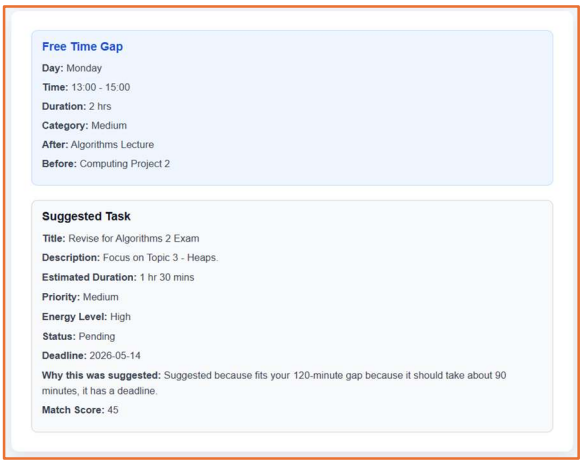


Image 1 has a Match Score of 50 whilst Image 2 has a Match Score of 45. Although the task's duration fits into both gaps, the system identifies that the gap in Image 1 is larger making it more suitable.

10.8 Common User Notes

Important usage notes include:

- task title and estimated duration are required
- timetable title, day, start time and end time are required
- start time must be earlier than end time
- invalid input produces an on-screen error message
- successful submissions update the visible lists
- recommendations are only generated from real gaps between sessions
- the active day window can be adjusted by the user
- the selected day window is remembered after refresh
- completed tasks are not recommended until they are marked pending again

11 REFERENCES

Express (n.d.) *Routing*. Available at: https://expressjs.com/en/guide/routing.html

Express (n.d.) *Using Middleware*. Available at: https://expressjs.com/en/guide/using-middleware.html

Express (n.d.) *Writing Middleware for Use in Express Apps*. Available at: https://expressjs.com/en/guide/writing-middleware.html

GitHub Docs (n.d.) *About repositories*. Available at: https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

MDN Web Docs (n.d.) *Client-side form validation*. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

MDN Web Docs (n.d.) *Using the Fetch API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs (n.d.) *Window: localStorage property*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

MDN Web Docs (n.d.) *<input type="date">*. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/date

Nielsen Norman Group (n.d.) *10 usability heuristics for user interface design*. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/

React (n.d.) *Conditional rendering*. Available at: https://react.dev/learn/conditional-rendering

React (n.d.) *useState*. Available at: https://react.dev/reference/react/useState

Shneiderman, B. (n.d.) *The Eight Golden Rules of Interface Design*. Available at: https://www.cs.umd.edu/users/ben/goldenrules.html

SQLite (n.d.) *CREATE TABLE*. Available at: https://www.sqlite.org/lang_createtable.html

SQLite (n.d.) *Date and Time Functions*. Available at: https://www.sqlite.org/lang_datefunc.html

SQLite (n.d.) *Foreign Key Support*. Available at: https://www.sqlite.org/foreignkeys.html

Vite (n.d.) *Getting started*. Available at: https://vite.dev/guide/

W3C Web Accessibility Initiative (n.d.) *WCAG 2 Overview*. Available at: https://www.w3.org/WAI/standards-guidelines/wcag/